

Report

A Controlled Social Media Simulation Environment for Psychological Research

Mika Koch
Computer Science

Studies from the School of Science and Technology at Örebro University
Örebro 2026

Studies from the School of Science and Technology at Örebro University



Mika Koch

A Controlled Social Media Simulation Environment for Psychological Research

Supervisor: **Todor Stoyanov**

Examiner: **Andrey Kiselev**

© Mika Koch, 2026

Abstract

Social media platforms have become a central part of everyday life and a standard interaction format for communication and information consumption. Their familiar interaction patterns make them a valuable instrument for fundamental research in psychology, providing an ecologically valid setting in which core psychological processes can be studied under controlled conditions.

However, existing tools that emulate social networks for psychological research are often limited by restricted customisability of content and interface, constrained access to fine-grained interaction data, and privacy concerns regarding data storage and ownership.

To address these limitations, this project introduces tablet-social-simulator, a React-based application that emulates an authentic social media feed while providing experimental control, privacy-preserving data handling, and structured behavioural data collection. The application was developed through an iterative, client-centred design process in which key design decisions were validated and refined across multiple feedback cycles.

To support both primary user groups, researchers and participants, the system is split into two interfaces: a tablet-based participant interface that presents a Facebook-inspired feed with text, image, and video posts, and a browser-based researcher interface for authoring and managing study content as well as inspecting and exporting study data.

The evaluation indicated that the simulator was perceived as highly usable and a realistic approximation of an actual social media environment, supporting intuitive and natural participant behaviour. In addition, the researcher admin interface enabled non-technical users to configure highly controlled studies and to export structured behavioural data for analysis.

Keywords

Social Media Simulation Software (SMSS), Behavioural Logging, Dwell Time, Researcher Authoring Interface, React, Capacitor (Android), WebDAV Storage

Contents

1 Introduction **4**
- 1.1 Problem Formulation . . . 4
- 1.2 Project Goals and Success Criteria . . . 5
- 1.3 Scope & Delimitations . . . 5
- 1.4 Terminology . . . 6
- 1.5 Division of Work . . . 6
- 1.6 Outline . . . 7

2 Background & related work **8**
- 2.1 Methods for Studying Social Media Behaviour . . . 8
  - 2.1.1 Real Platform Studies . . . 8
  - 2.1.2 Surveys and Vignette Studies . . . 8
  - 2.1.3 Open-source social media simulation software (SMSS) . . . 8
  - 2.1.4 Ethical advantages of SMSS compared to real-platform studies . . . 9
  - 2.1.5 Behavioural measurement from simulator logs . . . 10
  - 2.1.6 The Emergence of Generative Agent-Based Modeling (GABM) . . . 10

3 Design **11**
- 3.1 Design objectives and constraints . . . 11
- 3.2 Requirements Engineering . . . 11
  - 3.2.1 User Personas, User Stories, and User Scenarios . . . 11
  - 3.2.2 Functional & non-functional requirements . . . 13
- 3.3 Iterative Design Process . . . 14
  - 3.3.1 Iteration 1: Concept to Prototype . . . 14
  - 3.3.2 Iteration 1: Concept to Prototype . . . 15
  - 3.3.3 Iteration 2: Prototype to a refined system . . . 15
  - 3.3.4 Iteration 3: Consolidation and Final Adjustments . . . 16

4 Implementation **19**
- 4.1 Technology Stack . . . 19
  - 4.1.1 Frontend Framework: React.js . . . 19
  - 4.1.2 Native Container: Capacitor . . . 20
  - 4.1.3 Storage: WebDAV server . . . 20
- 4.2 System Architecture . . . 20
  - 4.2.1 High level component view . . . 20
  - 4.2.2 Unified Codebase Strategy . . . 22
  - 4.2.3 Component view . . . 22
- 4.3 Participant-Facing Tablet Interface . . . 23
  - 4.3.1 Session flow . . . 23
  - 4.3.2 Rendering posts, comments and social signals . . . 26
  - 4.3.3 Behavioural logging . . . 27

4.4 Researcher Admin Interface . . . 27
4.4.1 Researcher workflow . . . 28
4.4.2 Storyline authoring . . . 28
4.4.3 Previewing Storylines . . . 30
4.4.4 Consent and SurveyIntro settings . . . 32
4.4.5 Analytics and export . . . 32
4.5 Data Storage, Synchronisation and CSV Export . . . 33
4.5.1 File structure . . . 34
4.5.2 WebDAV-based cloud storage . . . 35
4.5.3 CSV export . . . 35
4.6 Deployment . . . 36
4.6.1 Installing the tablet application (Android) . . . 36
4.6.2 Deploying the researcher admin interface (web) . . . 36
4.6.3 WebDAV configurations . . . 37

5 Evaluation and Validation **38**
5.1 Participant Interface Pilot . . . 38
5.1.1 Methodology . . . 38
5.1.2 Results and Feedback . . . 38
5.2 Client Evaluation: Admin Dashboard . . . 39
5.2.1 Methodology . . . 39
5.2.2 Interview Findings and Feedback . . . 39
5.3 Data Output . . . 39
5.4 Heuristic evaluation (Nielsen's 10 usability heuristics) . . . 41
5.4.1 Participant-facing tablet interface . . . 41
5.4.2 Researcher admin dashboard . . . 42

6 Conclusions **43**
6.1 Review of project goals . . . 43
6.2 Summary of contributions . . . 44
6.3 Critical reflection and limitations . . . 44
6.4 Future work . . . 45

References **47**

Chapter 1
Introduction

Social media platforms have become a central interaction format for communication and information consumption, which has increased their relevance for psychological research. This creates a need for controlled experimental environments that can recreate key interaction patterns of social-media use while allowing researchers to control what participants are exposed to and to measure behaviour during exposure. However, existing approaches for creating such environments have limitations that reduce their suitability for rigorous psychological studies.

1.1 Problem Formulation

This project designs and implements a social-media simulator that emulates a realistic feed experience while enabling deterministic and reproducible stimulus presentation, precise manipulation of social signals, fine-grained behavioural logging, and privacy-preserving data handling under institutional control.

Current approaches rarely provide this combination in a practical form for rigorous studies.

Real social media platforms such as Facebook, Instagram, X, and Reddit offer a highly familiar user experience and can therefore elicit realistic participant behaviour. At the same time, researchers have limited control over content and its presentation, including the ordering of posts and comments and the insertion of advertisements. This limits reproducibility, because identical exposure to the same stimuli cannot be guaranteed across participants. Moreover, social signals such as reactions, like counts, and comment visibility cannot be precisely manipulated or measured, even though such manipulations are often central to the research question. Access to process data (e.g., time spent per post or reactions to comments) is either unavailable or heavily restricted. Finally, platform-specific data storage practices and third-party data sharing may conflict with strict academic requirements for privacy and data protection.

Beyond real platforms, no-code tools and app builders such as Bubble and Adalo appear to provide a convenient way to create simulated social-media environments, sometimes even offering templates resembling existing platforms. In practice, these tools come with significant constraints for research use: behaviourally relevant measures such as time spent on individual posts are often unavailable, and manipulating social signals can require complex workarounds or custom development. Researchers also have limited transparency and control over where and how data are stored due to closed hosting models, which complicates compliance with institutional privacy standards and migration to university-controlled infrastructure. For psychology researchers without a technical background, there is limited support for reliably building deterministic feeds, reproducing them identically across participants, and deploying them to Android tablets. These con-

straints make no-code platforms an unattractive option for supporting research centred around social network emulation.

1.2 Project Goals and Success Criteria

The project goals and corresponding success criteria are defined as follows. To characterise the intended maturity of the system, readiness is framed in terms of Technology Readiness Levels (TRL) [**?**], a scale originally developed by NASA that ranges from basic principles (TRL 1) to proven operation in the field (TRL 9).

G1: Lab-ready participant experience on Android tablets
Success: The simulator runs on an Android tablet app and supports a stable study session flow from consent to completion. In TRL terms, the target is TRL 6: the system is demonstrated in a relevant environment (i.e. a realistic lab session with representative content and participant interaction on the target hardware), but has not yet been validated through repeated independent deployments across multiple research groups (TRL 7+).

G2: Ecologically valid feed interaction
Success: Participants can naturally use a familiar Facebook-style feed interface, including scrolling through posts and interacting via reactions and comment threads across common media types (text, image, video).

G3: Researcher accessibility and workflow
Success: A non-technical researcher can set up a complete study, preview the participant experience, and export collected data using a researcher-facing interface, without modifying source code.

G4: Deterministic stimuli and experimental control
Success: Storylines can be configured as deterministic feeds with researcher-defined ordering and interaction settings, enabling identical exposure across participants and sessions.

G5: Fine-grained behavioural logging
Success: The system records structured behavioural events that capture both active interactions (e.g., reactions, comment interactions) and passive engagement measures (e.g., time spent per post).

G6: Analysis-ready export
Success: Behavioural logs can be transformed and exported as structured CSV datasets with consistent columns and encodings, suitable for direct use in external statistical analysis.

G7: Institutional data control and privacy
Success: Study assets and behavioural logs are stored using pseudonymous identifiers on researcher-controlled infrastructure, with data handling that supports institutional privacy and governance requirements.

1.3 Scope & Delimitations

The scope of this project is to design and implement a self-contained social-media-style simulator for use in psychological studies, where researchers can author predefined storylines consisting of posts and comment threads and present them to participants in a feed interface. The system focuses on providing a consistent participant flow and on collecting

behavioural logs such as scrolling, reactions, and dwell time. The intended use is small-scale, researcher-managed deployments where stimulus presentation and data capture can be configured and reproduced across participants and sessions.

The project is delimited in the following ways:

- The system does not replicate commercial-platform recommendation algorithms, dynamic ranking, or personalised feeds.
- Full social-network functionality is out of scope, including user profiles, direct messaging, and notification systems.
- The tool does not perform automated statistical analysis.
- The participant application is designed and optimised for Android tablets only.
- The system is not designed for public deployment at internet scale.
- The system is not a full survey platform; it integrates with surveys via handoff/links rather than replacing survey tooling.
- No personally identifying information is collected; the system is designed to comply with university privacy standards.
- The system does not support simultaneous multi-user interaction.

1.4 Terminology

Ecological Validity: The extent to which the experimental environment in a study approximates the real-world setting being examined. In this project, it refers to how closely the simulator mimics the "look and feel" of an actual social media platform.

Experimental Condition: A specific study setup defined by the researcher. In this project, conditions are typically implemented by assigning different storylines and/or settings to different participants.

Storyline: A predefined, scripted sequence of social media posts (text, images, video) and associated social signals (likes, comments).

Deterministic Stimuli: Stimuli that are presented in a fixed, researcher-defined form. In this project, deterministic stimuli means that the content and ordering of posts, comments, and social signals within a storyline are predetermined rather than generated or personalised during the session.

Social Signals: Visible indicators of engagement and social feedback associated with content, such as like counts, reaction counts, comments, and comment like counts.

Dwell Time: The amount of time a participant spends viewing a specific post or content item during the feed interaction.

1.5 Division of Work

This report was conducted as an individual project. All phases of the work—requirements analysis, design, implementation, and evaluation—were carried out by the author.

1.6 Outline

The rest of this report is organised as follows:

Chapter 2 reviews prior work on methods for studying social media behaviour and open-source social media simulation software (SMSS). It motivates the ethical advantages of simulator-based studies over real-platform approaches, discusses behavioural measurement from simulator logs, and positions the tablet-social-simulator with respect to derived design requirements.

Chapter 3 describes the design of the system, including design objectives and constraints, requirements engineering (personas, user stories, functional and non-functional requirements), and the iterative design process that shaped the final solution.

Chapter 4 explains the implementation, covering the technology stack, system architecture, the participant-facing tablet interface, the researcher admin interface, data storage and synchronisation, CSV export, and deployment.

Chapter 5 presents the evaluation and validation of the simulator. It reports findings from the participant interface pilot and the client evaluation of the admin dashboard workflow, verifies the produced data output (raw telemetry and exported CSV), and inspects usability through a heuristic evaluation.

Chapter 6 concludes the report by reviewing how the implementation meets the project goals, summarising the main contributions, reflecting on limitations and threats to validity, and outlining directions for future work.

Chapter 2
Background & related work

2.1 Methods for Studying Social Media Behaviour

Research on behaviour in social media environments has historically relied on two main methods: studies on real platforms and survey or vignette paradigms. A third methodological approach is the growing use of social media simulation software (SMSS).

2.1.1 Real Platform Studies

Field and trace-data studies on real platforms provide rich behavioural data [8]. However, these studies often face two practical constraints. First, platform restrictions can limit access to relevant data and make it difficult to test causal effects of recommendation systems and design features [10]. Second, legal and ethical requirements can conflict with platform privacy policies and data-sharing practices, which creates challenges for compliant data handling [8].

2.1.2 Surveys and Vignette Studies

Survey and vignette studies replace real interaction with hypothetical scenarios. Participants may view screenshots or text descriptions of posts and then report how they would react to other users, content, or policies [10]. These designs are attractive because they are inexpensive, scalable, and allow tight control over stimuli and conditions. Their limitation is that they measure reported responses rather than observed behaviour on platforms. Evidence suggests that self-reports correlate only moderately with logged digital media use, which means self-reports can be an imprecise proxy for actual behaviour [19]. Similar concerns have been reported for ESM-based measures (Experience Sampling Method; repeated in-the-moment self-reports prompted throughout the day) [24].

Together, restricted platform access, ethical constraints, and the limits of self-report have motivated methods that combine naturalistic interaction, direct behavioural logging, and strong experimental control [10]. Social media simulation software (SMSS) addresses this methodological gap by providing controlled, interactive environments while enabling structured behavioural measurement.

2.1.3 Open-source social media simulation software (SMSS)

Social Media Simulation Software is defined as a tool that allows researchers to collect behavioural data on how participants engage with content specified by the researchers, using interactive interfaces that mimic real social media platforms [10].

Key established tools include:

The (Mis)Information Game (MG)

It provides a customisable environment to study how participants process and spread misinformation within a simulated social media platform. It offers researchers high customisation and behavioural measurement, measuring not only active responses but also dwell times, and it supports high usability by not requiring programming skills because studies are authored via a Google Sheets template [5]. A key feature is the ability to implement a customised dynamic feedback system, e.g., changing the participant's follower count based on interactions with posts.

However, it comes with notable constraints for psychological research: it requires external hosting through Google Firebase for hosting and data storage, which comes with technical setup work and potential cost and privacy concerns; it does not support video content [5]; and the UI resembles earlier-generation social media layouts, which may reduce ecological validity.

Mock Social Media Website Tool (MSMWT)

This tool prioritises ecological validity by focusing on high perceived realism. It is highly effective for replication of user interfaces of real platforms like Facebook or X and allows participant profile registration and the simulation of engagement from social contacts. It collects detailed behavioural data, supports video content, and is designed to be researcher-friendly through the use of spreadsheets for input and output [9, 10]. It must be integrated or deployed into infrastructure chosen and managed by the research team.

Truman Platform (TP)

Originally focused on cyberbullying research, with the main strength of simulating dynamic social feedback through the possibility for researchers to control and schedule both engagement (likes, replies) and associated notifications [22]. Similar to MSMWT, TP allows profile registration, but also enables participants to view, report, and block profiles, enabling research on content moderation [22, 10].

A significant drawback is that study configurations require modification of files in the source code [10], creating challenges in accessibility. Furthermore, the platform's utility is limited by its inability to support video content, and the duration of sessions has to be predefined, which potentially undermines the goal of simulating natural behaviour.

Across MG, MSMWT, and TP, several strengths can be identified, such as naturalistic interfaces, manipulation of platform features, and structured logging of behavioural data. However, these tools also share general limitations, including dependence on external hosting infrastructure and limited media support. In addition, none of these tools provides a self-contained tablet application that operates without reliance on web hosting or external services, which can be relevant for ecological validity given that contemporary social media use is commonly mobile and app-based.

Overall, existing tools demonstrate the value of custom social media environments for psychological research, but also reveal a methodological gap that motivates new solutions.

2.1.4 Ethical advantages of SMSS compared to real-platform studies

An important advantage of Social Media Simulation Software (SMSS) is that it allows researchers to study social-media-like interaction under bounded, reviewable conditions. In real-platform studies, the researcher typically cannot fully control what participants are exposed to, how the platform processes data in the background, or whether an intervention affects non-participants through network spillovers. Ethics guidance for internet research and digital behavioural data highlights that these contextual uncertainties make it harder to assess risk, align with participant expectations, and justify proportionality in ethics review [2, 3].

SMSS reduces these issues because the experimental environment is researcher-authored and institution-governed. First, exposure risks are easier to control: the feed content can be pre-screened and constrained to what is necessary for the research question, which supports the general ethical principles of minimising harm and avoiding unnecessary intrusion [17, 16]. Second, SMSS reduces bystander concerns. Real-platform field experiments can influence not only enrolled participants but also others connected to them; user concerns about consent, manipulation, and legitimacy have been documented empirically in public attitudes toward social media field experiments [21]. Highly visible controversy around platform-scale feed manipulation further illustrates how ethically sensitive it is when interventions occur in live environments where meaningful participant understanding is difficult to ensure [11, 25]. In contrast, an SMSS study can typically be limited to consenting participants within a controlled session, which makes boundaries and responsibilities clearer [2].

A third ethical advantage is data governance. With SMSS, researchers can decide exactly what is logged, where it is stored, and how access and exports are controlled, rather than inheriting a platform's opaque data ecosystem. However, this only provides an ethical benefit if logging is designed responsibly. The EDPB emphasises that pseudonymisation is a safeguard, not anonymisation: pseudonymised logs can remain personal data if re-identification is reasonably possible via additional information or linkage [7].

Overall, SMSS offers an ethical advantage over real-platform studies because it enables researchers to keep both exposure and data collection proportionate, transparent, and institution-controlled, while reducing spillover effects and the legitimacy problems associated with covert or difficult-to-explain platform interventions [2, 3, 21, 7, 6].

2.1.5 Behavioural measurement from simulator logs

Simulator-based studies become methodologically stronger when they treat the simulator as a measurement instrument, not just a stimulus delivery tool. The main advantage is that simulators can produce behavioural process data: time-stamped traces showing what participants were exposed to and how they interacted with that exposure over time.

A common simulator metric is dwell time. Dwell time is valuable because it captures passive consumption even when no explicit action occurs, but it is also ambiguous: long durations can reflect careful processing, confusion, distraction, or idle time. For this reason, research on engagement metrics argues that dwell time should be complemented with visibility/viewport context—logging whether content was actually visible and for how long—to better distinguish brief "pass-by" exposure from deeper engagement patterns [12].

Finally, behavioural measurement is not only about what is logged, but about whether the logs are analysis-ready and defensible. This requires clear definitions, stable identifiers linking events to specific stimuli, and systematic validation of whether outputs behave as intended under the study's assumptions [20].

2.1.6 The Emergence of Generative Agent-Based Modeling (GABM)

A recent trend in this research area is the rise of Generative Agent-Based Modeling (GABM). In GABM, researchers combine agent-based models with large language models (LLMs) to generate posts, replies, and social behaviour in a synthetic platform environment. The main idea is not to measure real participants, but to explore how different platform rules and interventions might shape system-level outcomes at scale.

At the same time, an open challenge is realism and validation. Even when generated content looks plausible, that does not guarantee that behaviours or network dynamics match those of real users. Credible use of GABM therefore depends on transparent assumptions, careful evaluation, and clear limits on what conclusions can be drawn from synthetic behaviour traces [14, 20].

Chapter 3
Design

This chapter describes how the tablet-social-simulator was designed for psychological research use, and how the system evolved through three rounds of feedback with the client at the psychology faculty. The focus is on what the system is intended to do and why key design decisions were taken.

3.1 Design objectives and constraints

The starting point for the design was the set of methodological requirements for experimental social-media simulators developed in the related work chapter: realistic user experience, strong experimental control, fine-grained process data, accessibility for non-technical researchers, and robust privacy and data protection. In addition, the project had several practical constraints that shaped the design:

Target platform and form factor: The tablet-social-simulator shall be designed and tested for use on a Lenovo Tab M8 (4th Generation) Android tablet, which serves as the primary device for conducting participant sessions. This motivated the use of a React-based frontend packaged in a native Android container via Capacitor.

Institutional constraints: University data-protection rules required that all behavioural logs could be stored on infrastructure controlled by the research group (e.g., WebDAV on a university server) and exported in a CSV format.

Project time frame: The system had to be designed, implemented, and brought to a pilot-ready state within approximately ten weeks.

3.2 Requirements Engineering

3.2.1 User Personas, User Stories, and User Scenarios

To elicit requirements, user personas can be useful. Personas are fictive users who represent key stakeholder groups and support user-centred design. Two personas were modelled based on discussions with the client:

- **Researchers:** psychology students or staff who design and run studies using the simulator.
- **Participants:** individuals taking part in those studies, interacting with the simulated feed on lab tablets.

Persona A (Researcher)

Name: Emma Larsson

Age: 24

Role: Second-year master's student in psychology (researcher)

Technical skills: No formal programming training; experienced with online survey tools and spreadsheet software

Context and goals

- Designing a lab experiment on how social feedback (likes, reactions) on comments influences people's opinions about a topic.
- Create social-media-like feeds for her study.
- Manipulate social signals (for example, high vs. low like counts).
- Run sessions with groups of participants.
- Collect and analyse behavioural data in a CSV file.

Needs for a future tool

- Easy way to design feeds and export data.
- Easy way to run A/B tests with the same feed but small changes.

Pain points and constraints

- Anxiety about misconfiguring feeds and only noticing it after data collection.
- Limited time to design and run the study.

Persona B (Participant)

Name: Ben Müller

Age: 21

Role: Bachelor student in Business Analytics (study participant)

Technical skills: Uses multiple social media platforms daily

Context and goals

- Participates in studies at university for small rewards.
- Wants the study to be easy to understand and not take too long.

Needs for a future tool

- Simple instructions.
- Confidence that his data is anonymous and handled confidentially.
- Clear feedback about what to do and when the task is finished.

Pain points and constraints

- Interruptions or confusing UI elements risk degrading both his experience and the quality of collected data.

Based on the personas, user stories and scenarios were formulated to describe typical interactions in simple, natural language. These provided an early, non-technical description of what the system should do.

Scenario 1: Emma sets up a new study

This scenario leads to the following central user stories from the researcher perspective:

- As a researcher, I want to create a discussion post with multiple comments so that I can simulate an online comment section about a specific topic.
- As a researcher, I want to show the same feed to two groups of participants where only the like counts differ so that I can isolate the effect of perceived support on opinions.
- As a researcher, I want to preview each condition as a participant would see it so that I can check that the like counts and order of comments look realistic before running the study.

Scenario 2: Ben participates in the study

From this participant perspective, the following user stories emerge:

- As a participant, I want on-screen instructions so that I immediately understand what to do.
- As a participant, I want a discussion interface that feels similar to real comment sections so that interacting with it is intuitive.

3.2.2 Functional & non-functional requirements

The personas and scenarios translate the more general design requirements for experimental simulators introduced in Section 2.2 into project-specific requirements for the tablet-social-simulator.

Functional requirements

- The system shall allow researcher-users to create, duplicate, and edit feeds.
- The system shall allow researchers to manipulate social signals.
- The system shall offer a test mode in which researchers can run through a study as if they were participants.
- The system shall log key behavioural events during sessions (such as post reactions and view time).
- The system shall allow researchers to export the logged data for a study as a CSV file.

Non-functional requirements

In line with Section 2.2, the design of the simulator is further constrained by the previously defined non-functional requirements of realistic user experience, strong experimental control, accessibility for non-technical researchers, data quality and interpretability, and privacy and data protection.

In addition, the system is constrained by the following non-functional requirements:

Reliability and data integrity: The system shall maintain operational stability throughout a participant session, ensuring that all interaction telemetry is captured and successfully synchronised to the institutional backend without data loss or corruption.

Performance: The participant interface shall achieve fast media rendering and maintain high responsiveness during core interactions (scrolling and reacting) on the target hardware, avoiding disruptive delays that would reduce the naturalistic user experience.

Robustness: The system shall handle expected errors (e.g., temporary storage/synchronisation issues) gracefully without corrupting session data and prevent the deployment of invalid storyline configurations in the researcher interface.

Maintainability and extensibility: The system shall utilise a modular design that enables the seamless integration of new social media interaction types or updated experimental data models with minimal impact on the established application core.

Compatibility: The researcher-facing admin interface shall be easily deployable across different desktop devices and screen sizes and shall remain fully usable in standard modern web browsers through a responsive layout.

3.3 Iterative Design Process

The design was developed in close collaboration with the client across three iterations. This iterative approach allowed the project to correct misunderstandings early, incorporate new needs as they emerged, and focus effort on features with clear research value. The primary client was a psychology lecturer supervising student research projects.

The following subsections summarise the main iterations and show how concrete feedback from the client side was translated into design changes and revised requirements.

In addition to the requirements formulated in the previous subsection, further interviews with the client led to a set of more concrete, context-specific requirements:

- The participant interface shall adopt a feed layout that closely resembles a Facebook-style UI.
- The system shall support photo, video, and text-only posts as content types within a storyline.
- The system shall allow researchers to control and manipulate the order of posts within a storyline.

3.3.1 Iteration 1: Concept to Prototype

The first iteration aimed to validate the overall structure of the simulator rather than completeness of features. The central design hypothesis was that a clear separation between a user view and an admin view would satisfy the different needs of the main user groups.

The prototype consisted of a user select page followed by a story viewer page. The user select page allowed the experimenter to choose or enter a participant (Figure 3.1), and the story viewer presented a storyline selection step before the feed was displayed (Figure 3.2). After selecting a storyline, the participant feed presented posts in a scrollable feed with a basic interaction set (likes and a flat comment thread), as shown in Figure 3.3. On the admin page, admins could create simple storylines by defining posts with text and images, supporting early authoring and assignment workflows (Figure 3.4). At this stage, both the user and admin views were implemented as a desktop-only interface to enable rapid prototyping and testing.

A local server with WebDAV storage was introduced from the start, so that the system could later be pointed to an external cloud server without complicated redesign. Content types were restricted to text and image posts, which was sufficient to test the storyline concept. A first version of CSV export existed but did not yet produce usable data, and the entire system was implemented as a desktop-only, single-page application.

3.3.2 Iteration 1: Concept to Prototype

The first iteration aimed to validate the overall structure of the simulator rather than completeness of features. The central design hypothesis was that a clear separation between a user view and an admin view would satisfy the different needs of the main user groups.

The prototype consisted of a user select page followed by a story viewer page. The user select page allowed the experimenter to choose or enter a participant (Figure 3.1), and the story viewer presented a storyline selection step before the feed was displayed (Figure 3.2). After selecting a storyline, the participant feed presented posts in a scrollable feed with a basic interaction set (likes and a flat comment thread), as shown in Figure 3.3. On the admin page, admins could create simple storylines by defining posts with text and images, supporting early authoring and assignment workflows (Figure 3.4). At this stage, both the user and admin views were implemented as a desktop-only interface to enable rapid prototyping and testing.

A local server with WebDAV storage was introduced from the start, so that the system could later be pointed to an external cloud server without complicated redesign. Content types were restricted to text and image posts, which was sufficient to test the storyline concept. A first version of CSV export existed but did not yet produce usable data, and the entire system was implemented as a desktop-only, single-page application.

Requirements emerging from Iteration 1

- **CSV export**
  - Export numeric behavioural data.
  - Support per-participant rows (not only per post).
- **Comment interactions**
  - Support comment replies.
  - Support comment likes.
  - Provide an admin setting to enable or disable participant replies per study.
- **Session identity**
  - Remove re-selection of previous participants; each session shall start from a new pseudonymous participant identifier.
- **Consent**
  - Display a consent or information page before the storyline begins, which must be acknowledged by the participant before proceeding.

Iteration 1 confirmed that the split between user and admin views and the concept of configurable storylines were appropriate, as the prototype already supported a complete end-to-end flow from session entry through feed interaction and basic administration.

3.3.3 Iteration 2: Prototype to a refined system

The second iteration introduced a tablet app for participants and a desktop interface for researchers. The core features were now implemented: admins could create and edit storylines, the system logged relevant behavioural data during sessions, and a CSV export could be produced from the stored logs. Storage was moved from the purely local setup of Iteration 1 to a cloud server operated by the university.

Requirements emerging from Iteration 2

- **CSV structure**
  - Export one row per participant.
  - Include a clear explanation of numeric indices used in the export (e.g., reaction type indices).
- **Participant ID visibility**
  - After the storyline is completed and the finish button is pressed, display the participant's ID on the tablet so that survey answers can be linked to the correct participant.
- **Reaction set**
  - Support a `dislike` reaction type in addition to existing reactions.
- **Free-text privacy**
  - Do not store the free-text content of participant-entered comments; only log that a comment action occurred.

These insights shaped the consolidation and polishing work in the next iteration.

3.3.4 Iteration 3: Consolidation and Final Adjustments

The third cycle aimed to validate the design in realistic usage conditions, while revealing and resolving small but important issues that could have undermined usability and data quality in actual experiments.

Requirements emerging from Iteration 3

- **Configurable study texts**
  - Allow researchers to edit the consent text directly in the admin interface.
  - Allow researchers to configure the survey handoff link (e.g., to an external survey).
- **Preview correctness**
  - Display image captions in the admin preview exactly as they will appear in the participant view.
- **Reaction persistence**
  - Ensure that `dislike` reactions are stored correctly when added in edit mode after a storyline has been created.

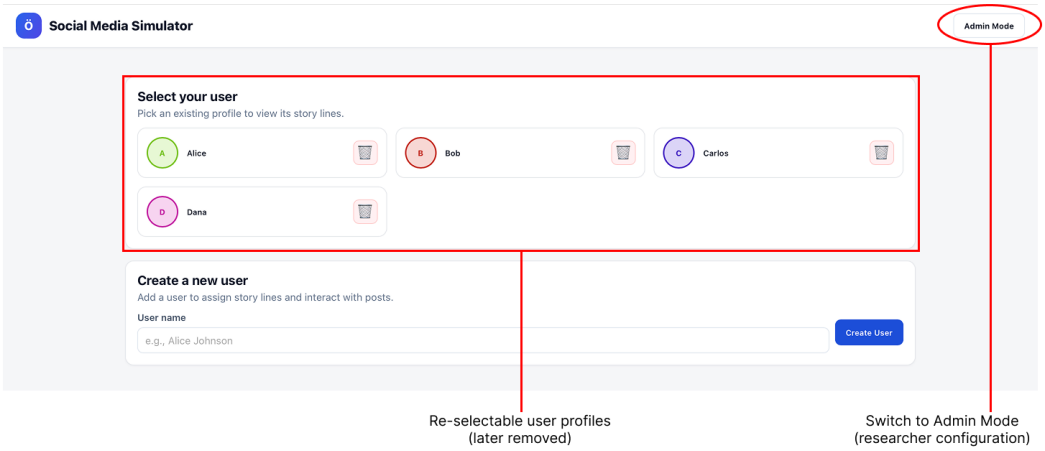


Figure 3.1: Iteration 1 entry screen of the prototype. Researchers could either select an existing user profile or create a new one to start a session. An Admin Mode button in the navigation provided access to a separate researcher-facing admin view (Split).

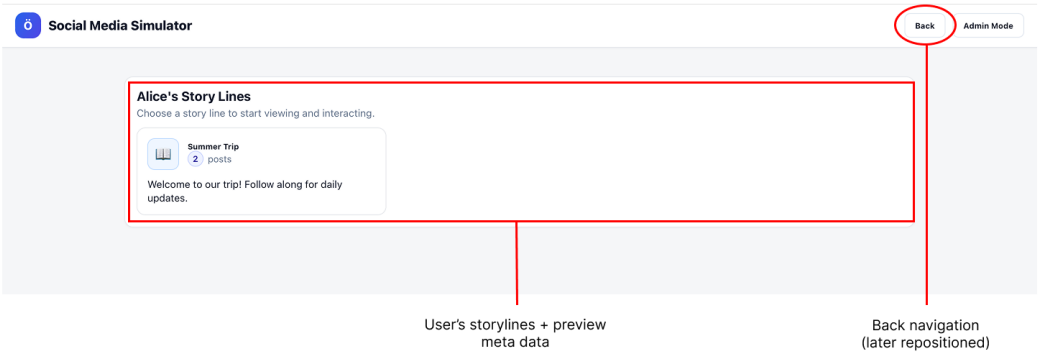


Figure 3.2: Iteration 1 storyline selection screen for a selected user. Available storylines were presented with brief preview metadata (e.g., number of posts and short description).

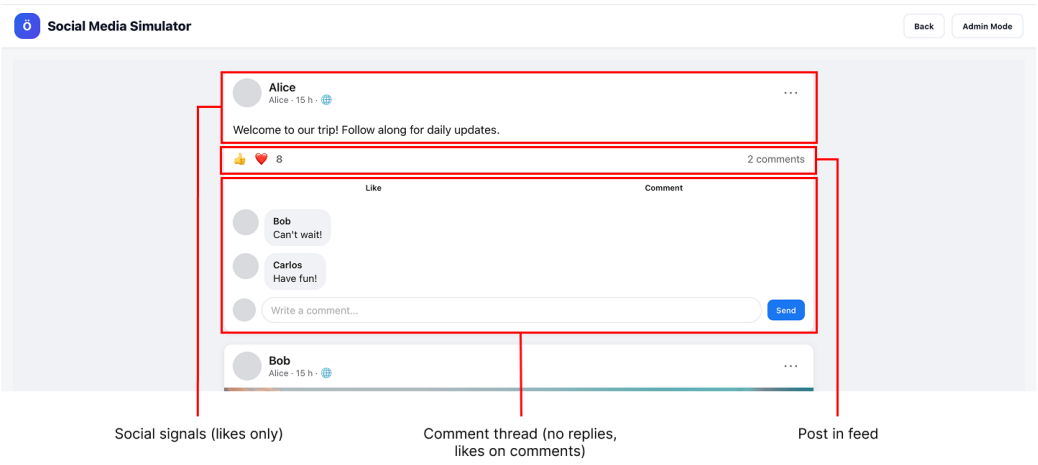


Figure 3.3: Iteration 1 participant feed view. Posts were displayed in a scrollable feed with basic social signals (likes only) and a flat comment thread (no replies and no likes on comments).

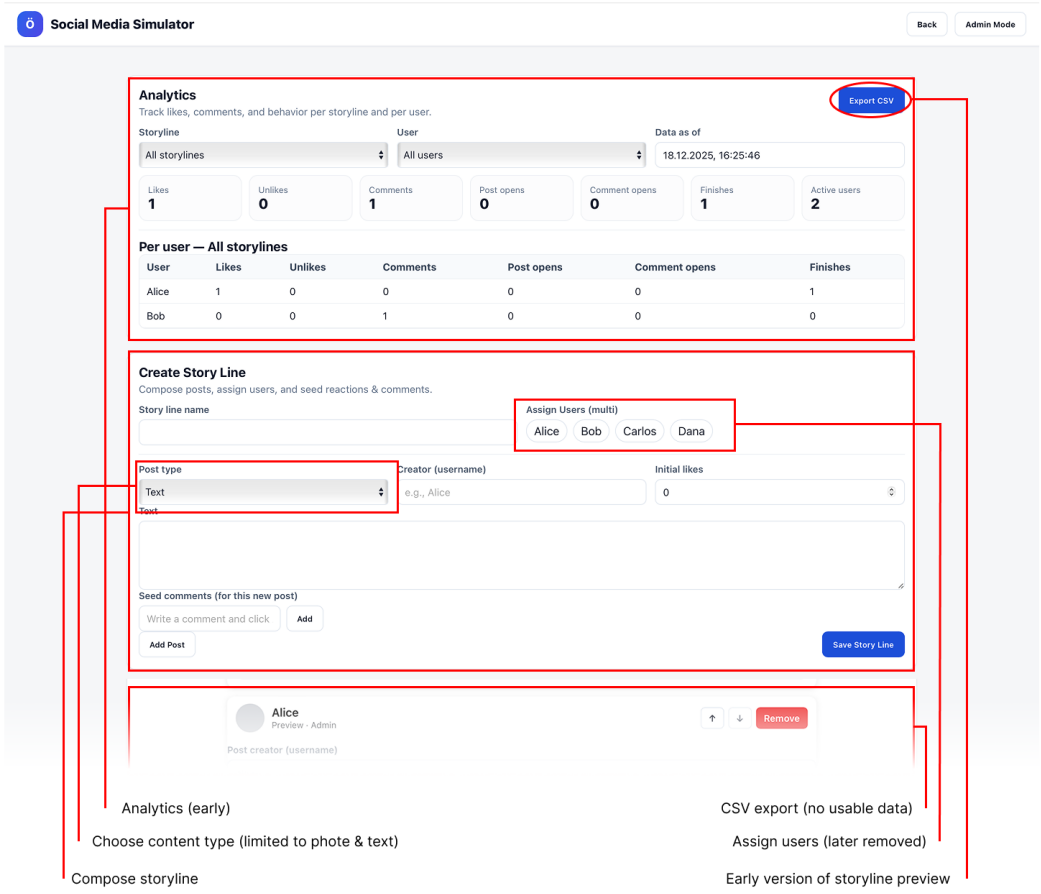


Figure 3.4: Iteration 1 researcher-facing Admin Mode. The interface supported early storyline composition (limited to text and photo posts), assignment of users to storylines, and an initial storyline preview. An analytics view with CSV export was implemented as a proof of concept, with exported data not yet analysis-ready in this iteration.

Chapter 4
Implementation

4.1 Technology Stack

4.1.1 Frontend Framework: React.js

The simulator is implemented as a web application using React. It was chosen because it supports a social-media-like interface while remaining maintainable and easy to extend for future studies.

Component-Based Architecture. The simulator is decomposed into React components such as ConsentPage, SurveyIntroPage, UserSelect, StoryViewer, and AdminPage. Each component encapsulates a clearly defined part of the workflow and reuses shared UI elements like AvatarCircle, which helps maintain a consistent look and interaction pattern across participant and admin views and keeps the overall structure easy to navigate and extend.

Behavioural Logging. React's state model and event handlers make it easy to track which screen is active, which posts are visible, and which reactions participants have given. Interactions such as scrolling, clicking, or playing a video are captured through these handlers and transformed into structured log entries, supporting the requirement for detailed, reliable behavioural data.

Deployment. The React application is built once and wrapped with Capacitor into an Android app for the lab tablets, while the same codebase also runs in a desktop browser for development and researcher use. This single-codebase strategy fits the project time constraints and simplifies testing, deployment, and maintenance.

Before selecting React, three options were evaluated against the project constraints and requirements.

1. Native Android Development. Implementing the simulator as a fully native Android app (e.g. in Kotlin/Java), or using cross-platform frameworks such as Flutter or React Native, would offer excellent performance and deep access to device features. However, this would either require integrating the researcher admin interface directly into the tablet app, which would complicate study setup and make configuration less comfortable on the small screen, or implementing it as a separate application, which would prevent code reuse between participant and admin views and increase overall complexity. These approaches also come with a steeper learning curve, which would have made it difficult to meet the roughly ten-week development timeframe.

2. HTML/CSS/JS. A lightweight approach using standard HTML, CSS, JavaScript was considered to reduce build complexity and dependencies. It could also have been packaged as an Android app using the same Capacitor-based approach described in Section 4.1.2, but the React-based solution fits the requirements better because it provides a clearer component structure and less complex state management for behavioural logging.

3. Other web frameworks. Other component-based frameworks like Vue.js or Angular were viable alternatives to React. They were not chosen primarily for pragmatic reasons rather than technical shortcomings: React has a strong ecosystem support, extensive documentation, and broad familiarity among students and researchers, which benefits maintainability.

4.1.2 Native Container: Capacitor

For the participant view the React app is packaged into an Android tablet application using Capacitor, which wraps the web app in a native container and exposes it as an installable app. From the user's perspective, the simulator behaves like any other installed Android app, with app icon, full-screen view, app switcher entry, while the implementation remains a web app. With this solution, the app establishes a bridge between the JavaScript runtime and the native Android layer which gives it access to device features like the filesystem.

Capacitor was chosen over exposing the simulator as a Progressive Web App (PWA), which would have made the app installable via the browser. This approach would have been technically less complex to develop, since it would avoid an Android build pipeline and tools like Android Studio. However, this option was not chosen because the simulator could not be permanently installed on the lab tablets without hosting it at a stable, always-accessible URL, or otherwise requiring a complex setup procedure (opening a local server, loading the URL, and "installing" the PWA) each time the app is used.

4.1.3 Storage: WebDAV server

Study configurations and behavioural log files are stored on a WebDAV server hosted on university infrastructure (`cloud.oru.se`).

WebDAV (Web Distributed Authoring and Versioning) is a standard extension of HTTP that makes a web server behave like a remote folder. In practice, files can be listed, downloaded, uploaded, and folders can be created over HTTPS in a way that resembles working with a normal filesystem. This makes it well-suited for this project because study artefacts (JSON + media) and logs can be stored as plain files on university infrastructure without introducing a database or a custom backend API.

Compared to building a custom REST backend or deploying a database, WebDAV reduced implementation and maintenance overhead while still supporting centralized storage, access control, and straightforward backups. This matched the project constraint of a short development timeframe and the requirement that data remains on institution-controlled infrastructure.

USB-based workflows were evaluated as an alternative for transferring files between devices, but they require manual copying, careful file naming, and constant checking that the correct logs have been transferred, and they carry a higher risk of data loss or mix-ups.

4.2 System Architecture

4.2.1 High level component view

The system can be understood as three cooperating components (Figure 4.1):

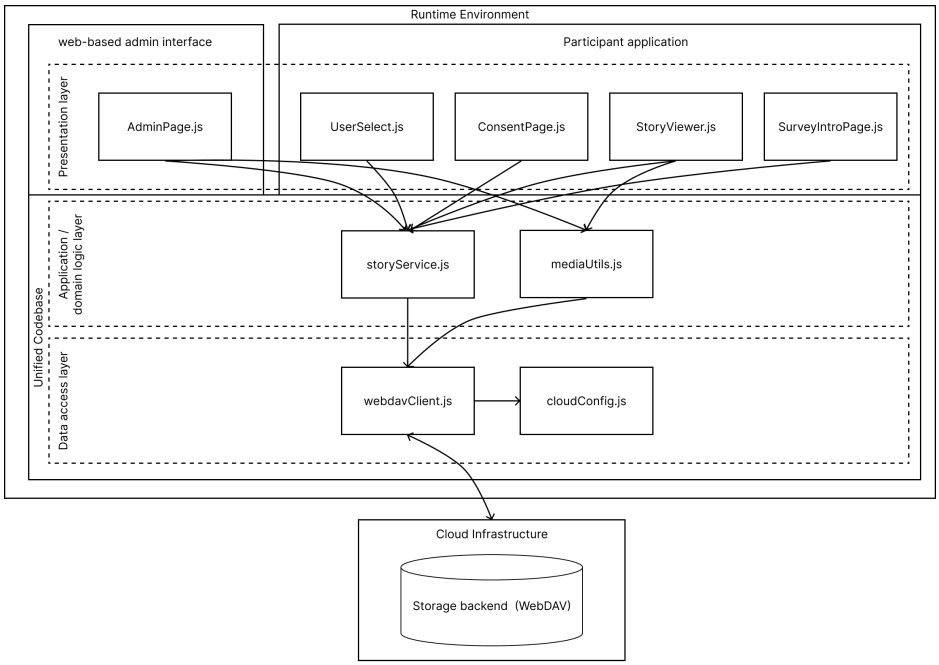


Figure 4.1: High-level component view of the simulator: both the researcher admin interface and the participant app share a common application layer that encapsulates story and media handling and accesses institutional cloud storage through a WebDAV-based storage component.

Participant application (Android App): The participant-facing application is a React application packaged into an Android app via Capacitor. The main participant components include ConsentPage.js, SurveyIntroPage.js, StoryViewer.js, and UserSelect.js. The tablet app focuses on a realistic feed experience and fine-grained behavioural logging

Researcher admin interface (web-based): Researchers access the admin interface through a browser on the clients laptop, implemented in React and centred around AdminPage.js. This view provides tools to create and edit storylines, configure posts and comments, set reaction options, and trigger CSV exports.

Storage backend (WebDAV): Both participant and admin views communicate with a WebDAV server hosted on university infrastructure (cloud.oru.se).

Internally, the React codebase follows an architecture organised into three layers (Figure 4.1):

Presentation layer This layer contains the React components that implement all user-facing screens and interface elements. In the participant application, this includes `UserSelect.js`, `ConsentPage.js`, `SurveyIntroPage.js`, and `StoryViewer.js`, which together guide participants through the session and render the simulated feed. In the researcher interface, `AdminPage.js` renders configuration forms, previews, and export controls.

Application / domain logic layer This layer captures the simulator's core concepts like storylines, posts, comments, participants, and events. The main module is `storyService.js`, which loads and saves study configurations, makes them available to the UI, and manages logging events. `mediaUtils.js` handles how images and videos are treated before they are shown in the interface. Any code that creates log entries or prepares CSV exports conceptually belongs to this layer.

Data access layer The data access layer is responsible for persistence and communication with external storage. `webdavClient.js` wraps all interaction with the WebDAV server, including authentication, folder management and file uploads, while `cloudConfig.js` centralises backend configuration such as base URLs and paths

4.2.2 Unified Codebase Strategy

Unlike traditional systems that maintain separate codebases for the mobile app and the admin dashboard, this project utilizes a Unified Codebase. The exact same React build is deployed to the Android tablets (wrapped in Capacitor) and served to researchers on the web.

Critical logic such as the data layer (`storyService.js`) and UI elements (`AvatarCircle`) is shared. This guarantees e.g. that the preview shown to researchers in the `AdminPage` is technically identical to the feed rendered in the `StoryViewer`.

4.2.3 Component view

The simulator is organised into three layers: presentation, application/domain logic, and data access. The following lists summarise the main modules and their responsibilities without relying on manual spacing or tab alignment.

Presentation layer

- **Participant UI components**
  - `UserSelect.js`: initial session setup (session label and storyline selection) and creation of a new pseudonymous session identifier.
  - `ConsentPage.js`: displays configurable consent text and gates the start of the session.
  - `SurveyIntroPage.js`: shows the participant ID and provides the survey handover after completion.
  - `StoryViewer.js`: renders the simulated feed (posts, comments, social signals) and forwards interaction events to the logging logic.
- **Admin UI components**
  - `AdminPage.js`: storyline authoring, settings editing, preview, analytics inspection, and CSV export controls.

Application / domain logic layer

- `storyService.js`: central service that loads and saves study configurations, exposes storylines to the UI, and provides a unified API for behavioural logging (e.g., scrolls, reactions, video plays).
- `mediaUtils.js`: shared helper functions for handling feed media (image/video), including prefetching and robust fallback behaviour.

Data access layer

- `webdavClient.js`: encapsulates WebDAV operations (list, read, write), authentication, folder management, and file uploads/downloads.
- `cloudConfig.js`: centralises backend configuration (e.g., WebDAV base URL and paths) so server details are not hard-coded in UI or domain logic.

4.3 Participant-Facing Tablet Interface

The participant-facing interface is implemented as a sequence of React screens that guide participants from consent through instructions to the simulated feed and finally to the end of the session. The main components are `ConsentPage.js`, `SurveyIntroPage.js`, `StoryViewer.js` and `UserSelect.js`.

4.3.1 Session flow

A typical session on the tablet proceeds as follows:

1. **Initialization (UserSelect.js).** At the start of a session, the researcher enters a custom session name (e.g. "GroupA_03") that can later be used to identify the session in exported data (Fig. 4.2a). At the same time, the system automatically generates a unique, pseudonymous session ID that is used internally to link all log entries for this run. This separates human-readable session labels from the internal identifier used to link log entries, while avoiding personally identifying information. On the same screen, the researcher selects which storyline should be shown on the tablet, before the tablet is handed to the participant.

2. **Consent and study introduction (ConsentPage.js).** ConsentPage loads a consent or information text specified by the researcher in the admin interface and displays it in a scrollable view (Fig. 4.2b). The participant can either confirm consent to proceed or use a back option to return to the previous screen, allowing the researcher to correct the session setup or abort the session if needed. Only after the participant confirms that they have read and accepted the information does the app proceed to StoryViewer, ensuring that no behavioural data is collected without consent.

3. **Simulation (StoryViewer.js).** The participant interacts with the simulated social media feed (Fig. 4.2c). This stage includes rendering posts, comments and social signals, enforcing condition-specific interaction rules, and generating behavioural logs. During this stage, interaction events are recorded continuously while the participant navigates the feed. Because this is the core of the simulator, the feed rendering and interaction mechanics are described in detail in Section 4.3.2, and the associated logging mechanisms are discussed in Section 4.3.3.

4. **Completion and handover.** When the participant presses the finish button in StoryViewer, the session transitions to SurveyIntroPage (Fig. 4.2d). This displays the participant's ID and instructions for proceeding to external questionnaires. At this point, pending log events are synchronised to the WebDAV server.

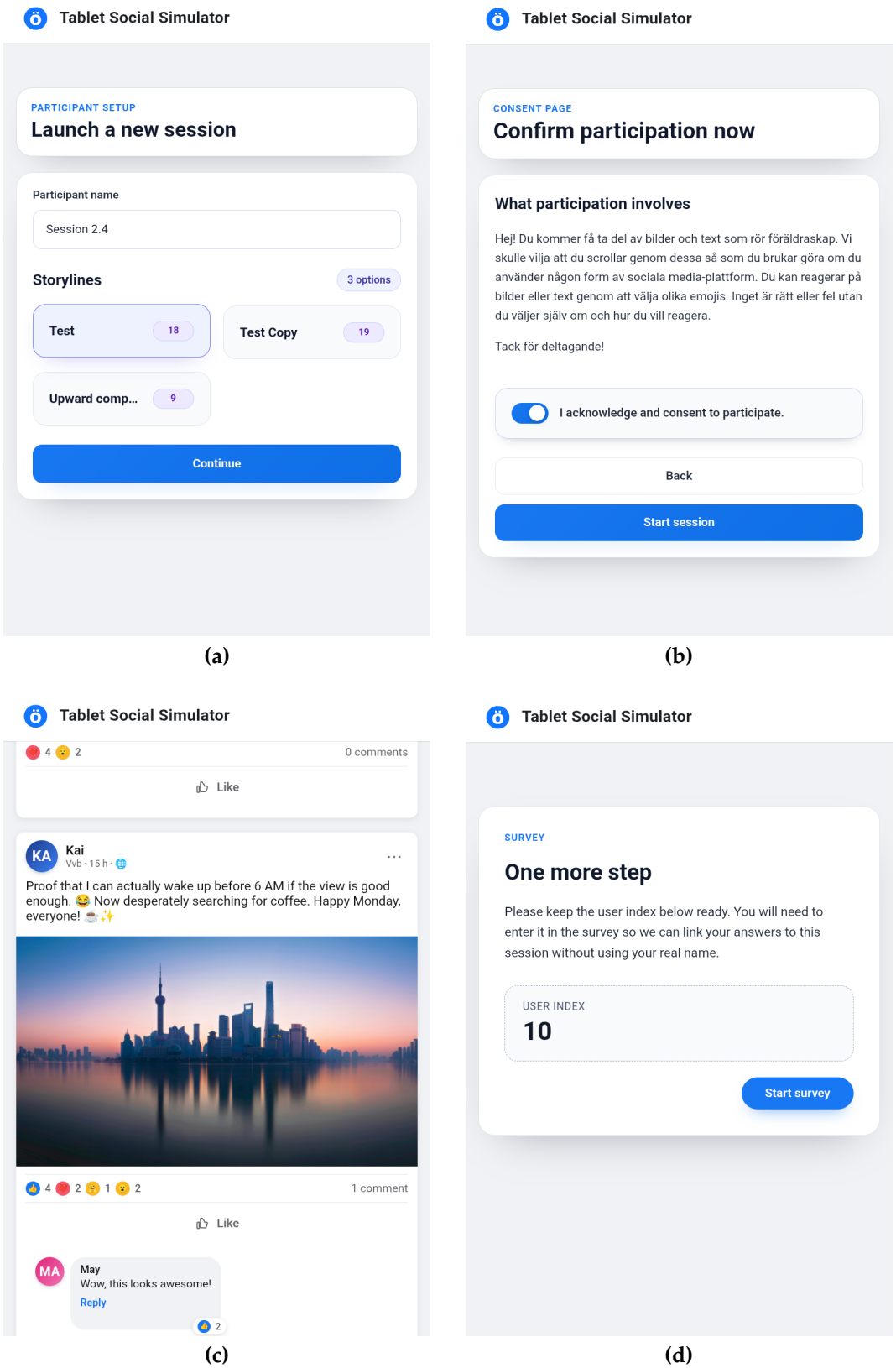


Figure 4.2: Screens of the participant-facing tablet interface. (a) Session initialization (UserSelect.js). (b) Consent and study information with back option (ConsentPage.js). (c) Example feed item in the simulated interface, showing post content, social signals, and comment interaction (StoryViewer.js). (d) Survey handover screen displaying the user index for pseudonymous linking (SurveyIntroPage.js).

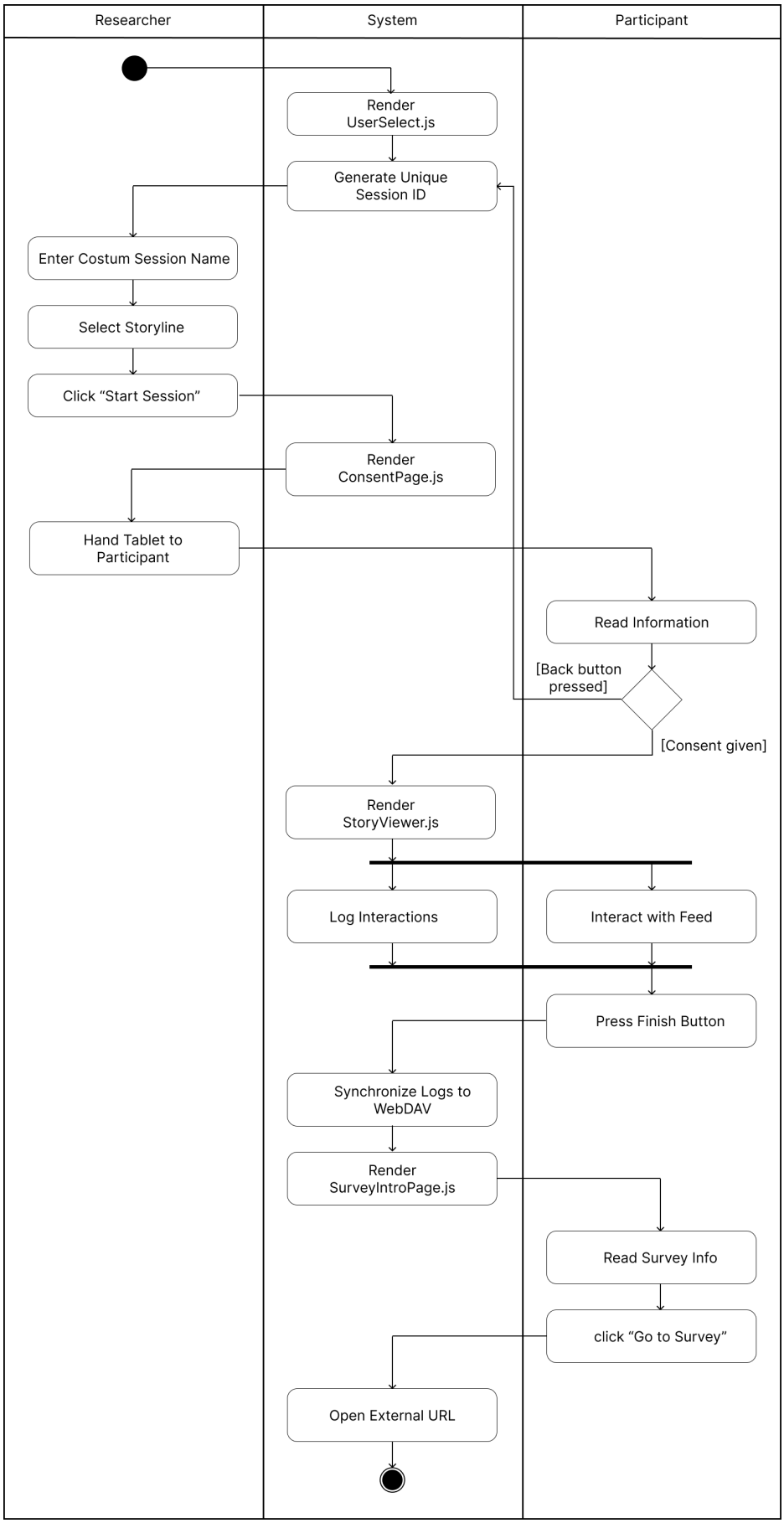


Figure 4.3: Participant session flow diagram. The researcher initializes the session on the tablet, the participant provides consent and completes the feed interaction while the system logs behavioural events, and finally the session data is synchronized to the WebDAV backend before the participant is forwarded to an external survey link.

4.3.2 Rendering posts, comments and social signals

The core of the participant experience is `StoryViewer.js`, which renders the simulated social-media feed. The selected storyline is passed into StoryViewer as a prop and represented as structured JavaScript objects (parsed from JSON earlier in the flow), containing post IDs, author metadata, timestamps, text content, media filenames/URLs, comment threads, and social signals such as like counts or reaction options.

Rendering is fully data-driven: each post object is mapped to a visual card with (1) a header (author, avatar, timestamp), (2) the content area (text and optional image/video), (3) a social signals row (reaction icons and counts, comment count), and (4) the comments section showing the discussion beneath the post (Figure 4.4). Comment threads and replies are rendered beneath each post, and interaction options such as liking, reacting, or replying can be enabled or disabled depending on the configuration created in the admin interface.

Comment discussions are rendered as a true thread rather than a flat list. A recursive function (e.g. `renderCommentTree`) renders a comment and then renders its replies in the same way, repeating this process for every reply level. This allows storylines to contain realistic nested conversations without imposing a fixed limit on reply depth.

Media is handled in a way that reduces dependence on live network requests during a session. Images and videos are pre-fetched as binary data (Blobs) and displayed using locally generated `blob:` URLs, rather than being loaded directly from remote URLs at the moment they appear on screen. In practice, this means that once an asset has been fetched it loads instantly from the device during scrolling and video playback remains smooth even if the network connection becomes slow or unstable.

This behaviour is implemented centrally in `mediaUtils.js`, which provides reusable helper functions for resolving media references into loadable URLs, applying the prefetching strategy, and handling failures. If a media file cannot be fetched, the UI falls back to a placeholder state while continuing to render the rest of the post normally, ensuring the session can proceed without broken layouts or interruptions.

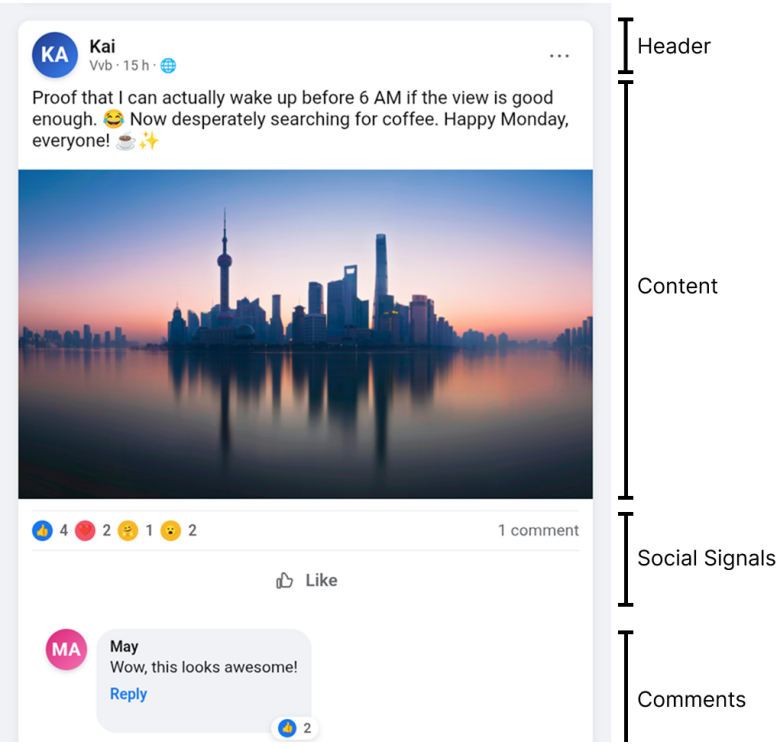


Figure 4.4: Structural breakdown of a feed item in the participant interface (`StoryViewer.js`).

4.3.3 Behavioural logging

`StoryViewer.js` records participant behaviour by converting each relevant interaction into a structured log event. The component detects interactions (e.g., tapping a reaction, liking a comment, scrolling) and forwards them to the central logging function `addLog(...)` in `storyService.js`.

Each logged event follows the same basic structure: a timestamp (`ts`), session identifiers (`user`, `storylineId`, `storylineName`), an action label describing what happened, and the affected `postIndex`. When extra detail is needed, it is stored in a `value` object (e.g., the chosen reaction type or the path of a liked comment).

Logging post reactions. In like-only mode, when the participant taps Like, the component toggles this state and immediately calls `storyService.addLog(...)` with either `action: "like"` (post becomes liked) or `action: "unlike"` (like removed), always including the corresponding `postIndex`.

In reaction-bar mode, a post can hold one specific reaction type (e.g., "love", "wow"). The viewer stores the currently selected reaction type per post. When the participant selects a reaction, the component updates this local selection and calls `storyService.addLog(...)` with `action: "react"`, storing the chosen type in `value.type`.

Logging comment reactions. Comment likes are tracked in the same two-step pattern: `StoryViewer.js` maintains local UI state and logs the change via the service. When a participant likes a comment, the viewer toggles the comment's liked state and calls `addLog(...)` with `action: "comment_like"`; when they remove it, it calls `addLog(...)` with `action: "comment_unlike"`. Because comment threads can be nested, the target comment is identified by a path in the thread, which is included as `value.commentPath` so that each logged event can be linked to the exact comment.

Logging time per post. The simulator records how long participants spend on individual posts. This is non-trivial because multiple posts can be visible at the same time while scrolling. To avoid double counting, `StoryViewer.js` assigns exposure time to a single "active" post at any moment.

The implementation uses an `IntersectionObserver` to track the visibility of each post element and compares their visible proportions (`intersectionRatio`). The post with the highest visibility is treated as the active post, provided it exceeds a minimum threshold (`intersectionRatio` $\geqslant 0.3$). When the active post changes (i.e., a different post becomes most visible), the system closes the previous viewing interval and logs a `time_on_post` event for the old post with the measured duration in milliseconds. Very short intervals are ignored (duration < 750 ms) to reduce noise from rapid scrolling.

This strategy provides a conservative and interpretable estimate of attention allocation: time is attributed to the post that most likely dominated the participant's screen, rather than being split across several partially visible posts.

4.4 Researcher Admin Interface

The researcher-facing admin interface is implemented primarily in AdminPage.js, designed for use in a desktop/laptop browser. In contrast to the participant UI, which prioritizes an ecologically valid feed experience and behavioural data collection, the admin view prioritizes experimental control and accessibility for non-technical researchers.

The unified-codebase strategy reuses core UI and domain logic across researcher and participant interfaces.

4.4.1 Researcher workflow

A typical researcher workflow proceeds as follows:

1. **Author or edit a storyline (Storylines tab).** The researcher either creates a new storyline or selects an existing one and defines/edits the content: an ordered sequence of posts, their associated social signals and scripted comment threads.
2. **Validation (Storylines tab/Preview tab).** The Preview view provides a higher-level validation by rendering the storyline in the same layout used for participants, making it easy to spot unintended behaviour.
3. **Configure consent and study introduction (Settings tab).** The researcher has the possibility to edit the consent text that participants see before the feed and adjust the instructions displayed on the SurveyIntroPage.
4. **Data Inspection/Export (Analytics tab).** After sessions have been conducted, the researcher uses the Analytics view to inspect collected data and export them in CSV format for analysis.

4.4.2 Storyline authoring

The Storylines tab is the main tool researchers use to build the stimuli shown in the participant feed. It is designed as a structured, form-based editor so that storylines can be authored without editing JSON manually or handling files on the server.

Conceptually, it combines three elements in one workspace: (1) a list of existing storylines, (2) a guided "builder" for storylines, and (3) a live preview that mirrors the participant feed.

Existing storylines list. At the top of the tab, the interface displays all existing storylines loaded from storage (see Fig. 4.5). For each entry, the researcher can Edit, Duplicate, or Delete. These actions are executed through the service layer (save, duplicate, delete) and followed by refreshing the list so the UI reflects the current backend state.

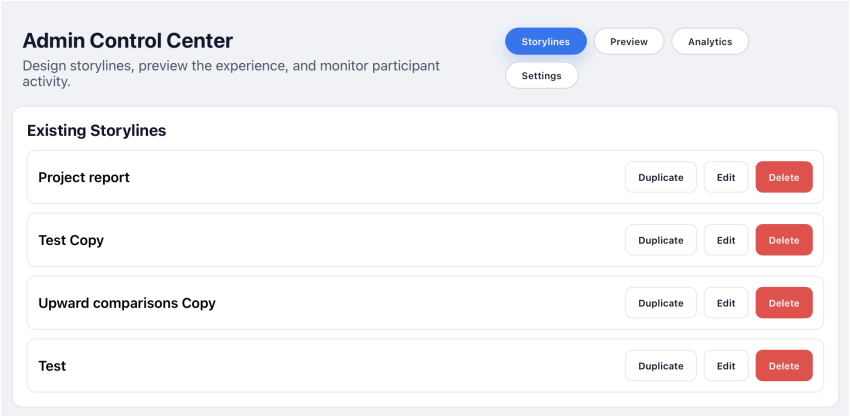


Figure 4.5: Storyline list view in the authoring tab, showing existing storylines loaded from storage and providing management actions (Edit, Duplicate, Delete).

Storyline-level settings. Before editing individual posts, the tab exposes a small set of global settings that control what participants will be able to do and what social feedback will be visible (see Fig. 4.6):

- **Reaction mode (reactionsMode):** either a simple "Like" interaction or a reaction bar with multiple reaction types.

- **Enabled reaction types (reactionsEnabled):** which reactions appear if the reaction bar is used.
- **Comments shown (commentsEnabled):** whether seeded comments are visible beneath posts.
- **Commenting allowed (commentingEnabled):** whether participants can write their own comments/replies.

These settings are intentionally placed at the top because they shape the rest of the authoring process. Only after these constraints are set does the user proceed to adding stimulus content. The "Next" button unlocks the post composer.

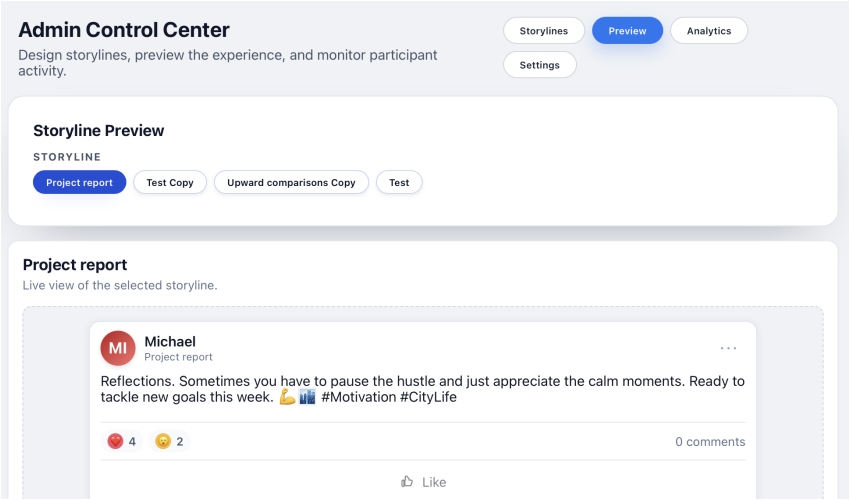


Figure 4.6: Storyline-level engagement settings controlling enabled reaction types, and comment visibility/writing permissions.

Post creation. New posts are created through an "Add Post" form with explicit fields for the elements that will later be rendered in the feed (see Fig. 4.7):

- Author metadata (name and avatar)
- Post type (text, photo, video)
- Text content (caption/body text)
- Media attachment (uploaded file for photo/video posts)
- Social signals (initial like count or per-reaction counts, depending on reaction mode)

The tab maintains these as "draft" state variables while the user is filling the form. When the researcher clicks Add Post, the drafts are assembled into a post object and appended to the current storyline's post list. For media uploads, the post stores a reference to the filename that will be persisted, while the actual File object is kept until saving so it can be uploaded together with the storyline configuration.

For each post, the Storylines tab also supports defining scripted comment threads (see Fig. 4.7). Comments are authored as structured entries (author, text, optional like count), and replies can be attached beneath a comment, enabling realistic "discussion" structures. Internally, comments are normalised and cleaned through helper functions (e.g., `coerceCommentEntry(...)` and `sanitizeCommentEntry(...)`) to ensure a consistent shape when stored and later rendered. To make editing reliable, each seeded comment is assigned a stable internal ID so that nested replies can be added, edited, or removed without the UI losing track of which node is being modified.

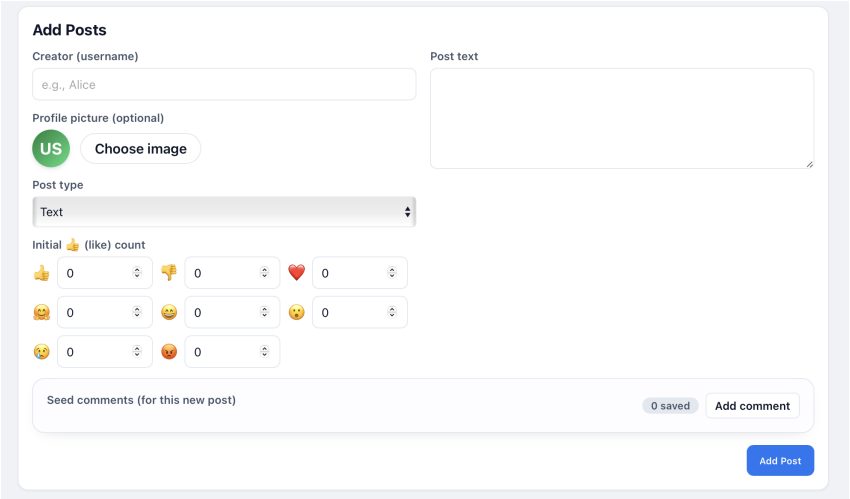


Figure 4.7: Post composer in the storyline editor, used to create new feed items by specifying author metadata, post type and content, media attachments, initial social signals, and seeded comment threads.

Live preview. Below the authoring forms, the tab renders a live preview that mirrors the participant feed layout (see Fig. 4.8). It functions as an immediate validation tool for researchers, allowing them to spot issues that are difficult to detect in raw form fields. The preview also supports quick editing actions:

- **Reordering:** The ordering of posts is implemented as an explicit order index field; drag-and-drop operations in the UI simply update this index.
- **Removal.**
- **Per-post editing:** via an expandable panel (`togglePreviewEditor`) where the researcher can adjust author name, caption/text, avatar, replace media, and reaction counts.

This makes storyline creation an iterative process: researchers can adjust content and immediately see the effect in a feed-like representation.

Saving. When saving, the editor performs lightweight validation (storyline name present, at least one post) and then assembles a clean storyline object plus a file bundle. Filenames are sanitised to avoid invalid paths, and the final storyline JSON plus files are passed to the service layer (e.g., `saveStoryline(...)`) for persistence on the WebDAV backend.

4.4.3 Previewing Storylines

Similar to the Live preview section in Storylines tab the Preview tab provides a dedicated validation view that renders a selected storyline in a feed-like layout (see Fig. 4.9). Compared to the live preview function embedded in the Storylines tab, this view is intentionally read-oriented: it supports quickly switching between storylines and verifying that global constraints (reaction mode, enabled reaction types, comment visibility, and participant commenting permissions) match the intended participant experience. Because the admin and participant views share the same codebase, data model, and media handling, the Preview tab closely matches the participant feed; the main difference is that the participant view additionally attaches logging handlers through the service layer. It encourages researchers to validate the final stimuli and helps catch errors before running participants.

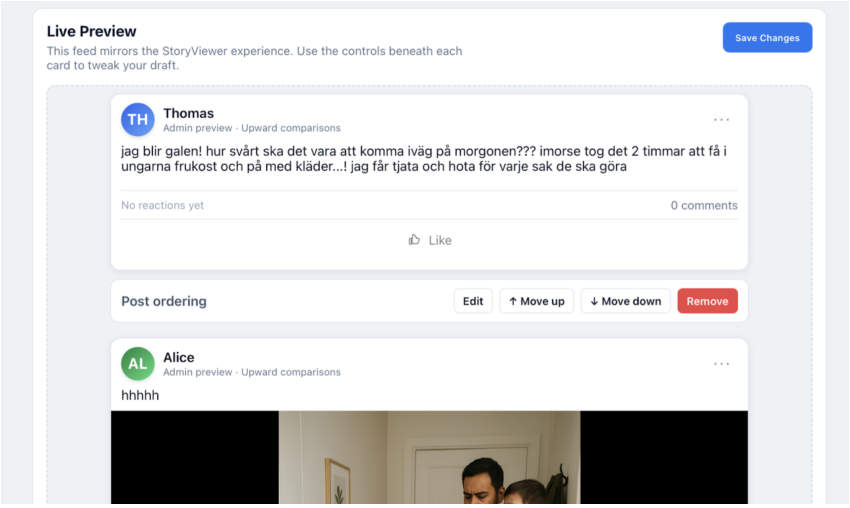


Figure 4.8: Live preview of the assembled storyline, rendering posts in a feed-like layout and supporting iterative editing actions such as reordering, removal, and per-post adjustments before saving.

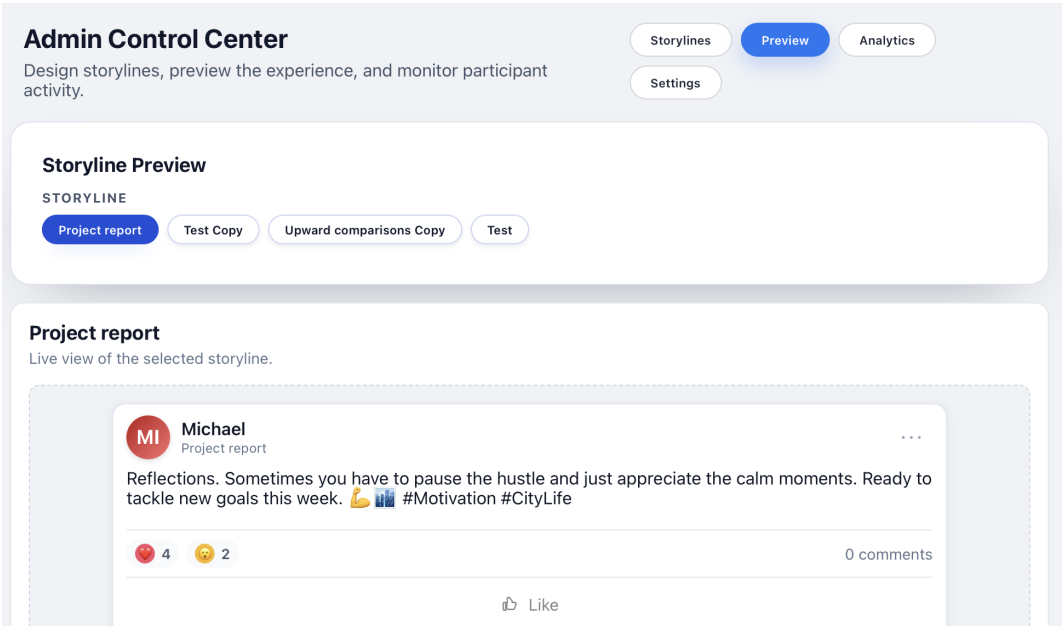


Figure 4.9: Preview tab in the admin interface, showing a live rendering of a selected storyline in the same feed layout as the participant view, enabling researchers to validate content, social signals, and overall presentation before running a session.

4.4.4 Consent and SurveyIntro settings

In addition to authoring the feed content itself, researchers are able to configure the texts that appear before the simulated feed in `ConsentPage` and after in `SurveyIntroPage` and configure the external URL opened after the storyline is finished (see Fig. 4.10).

These are managed in the Settings tab of `AdminPage.js`. This means the simulator can be reused across studies with different consent requirements or surveys, and researchers can update consent or instruction text without modifying the feed stimuli.

These settings are handled like other study configuration data in the admin interface: the Settings tab updates local React state in `AdminPage.js`, and saving triggers the same persistence mechanism. The configurations are stored in the `app-settings.json` object.

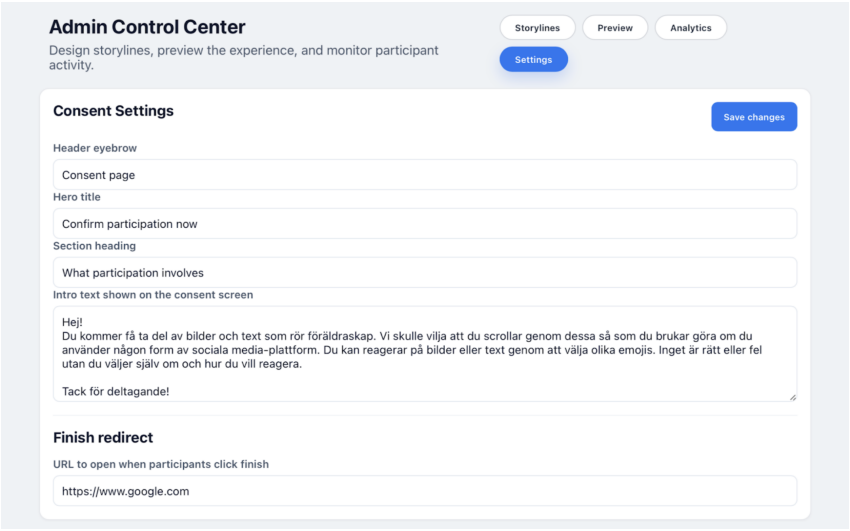


Figure 4.10: Settings tab in the admin interface used to configure participant-facing consent text and the finish redirect URL (e.g., an external survey link), enabling study-specific configurations.

4.4.5 Analytics and export

The Analytics tab in `AdminPage.js` is the researcher's entry point for inspecting collected interaction logs and exporting them in a structured format for statistical analysis. In contrast to the Storylines and Settings tabs (which author configurations), this view is read- and export-oriented. The actual CSV generation and storage/export pipeline are described in Section 4.5.

On startup, the admin interface loads the available storylines (`fetchStorylines()`) and the stored behavioural logs (`fetchLogs()`). The Analytics tab then lets the researcher select a storyline ID as an analysis scope (Fig. 4.11). All derived measures in the table are computed only from log entries whose `storylineId` matches the current selection, ensuring that exports reflect a single storyline at a time.

Users × Posts summary table. The central element of the tab is the "Users × Posts" table (derived in `perUserPost`, Fig. 4.11). For each participant and post position, it aggregates export-relevant measures, including:

- **Reaction outcome per post:** stored as a normalised reaction type key and later mapped to a numeric index.
- **Time on post:** computed by summing logged `time_on_post` intervals (`value.durationMs`) per post.
- **Liked comments:** derived by collecting comment-like events and resolving them to stable numeric comment identifiers.

Stable indices and reference tables. Because many statistical workflows prefer numeric encodings, the Analytics tab maintains deterministic index mappings (Fig. 4.12):

- `userIndexMap`: assigns stable indices to session labels/users (kept stable across runs by caching existing mappings and appending only new ones).
- `storylineIndexMap`, `contentTypeIndexMap`, and `reactionTypeIndexMap`: convert storylines, content types, and reaction types into numeric codes (with `none = 0` for reaction type).
- `commentIndexData`: enumerates seeded comments (including nested replies) in a deterministic order and assigns each a unique numeric identifier.

The tab displays these mappings in an "Index reference" panel and can export them as a separate CSV ("Export indices CSV") to serve as a compact codebook for later analysis.

Data management actions. Researchers can delete logs for a single user (within the selected storyline scope) or clear all analytics data. These actions call service functions such as `deleteLogsForUser(...)` and `clearLogs()`, and the UI refreshes by reloading logs (`fetchLogs()`) so the table immediately reflects the updated backend state.

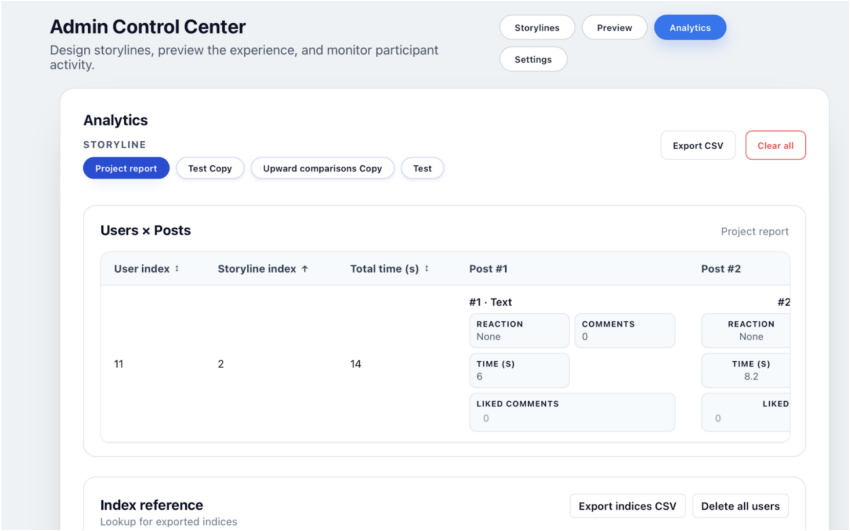


Figure 4.11: Analytics tab in the admin interface, providing storyline-scoped inspection of behavioural logs through a "Users × Posts" summary table and controls for exporting CSV data or clearing stored analytics.

4.5 Data Storage, Synchronisation and CSV Export

The storage layer supports the simulator's privacy and data-quality requirements by persisting study artefacts (storylines, media, settings) and structured behavioural logs on controlled institutional infrastructure. The institutional WebDAV backend is used as a remote filesystem with a small, inspectable directory structure, avoiding the need for a database service.

Figure 4.13 summarises the resulting data flow between the participant tablet application, the researcher admin interface, and the WebDAV backend. During authoring, the admin interface stores storyline configuration files in storylines/, uploads assets to media/, and writes study-level configuration (consent and survey handoff URL) to appsettings.json; participant labels are maintained in users.json. During sessions, the tablet

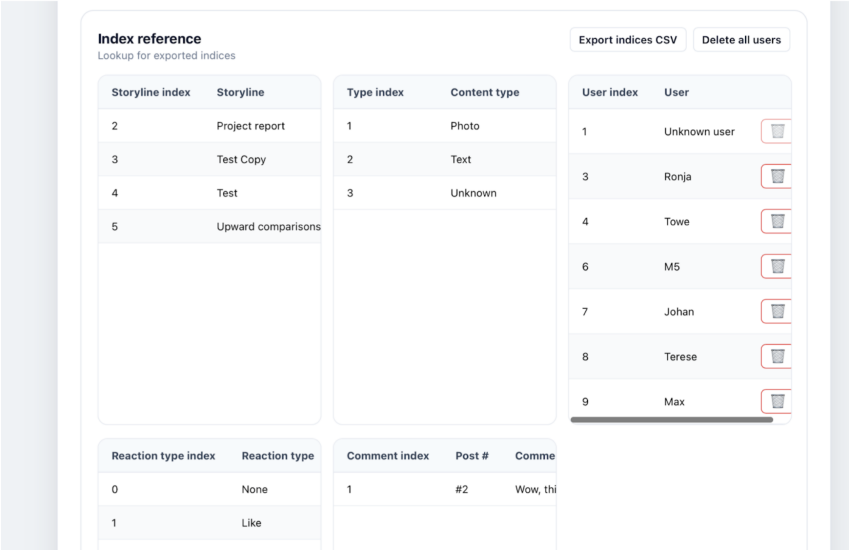


Figure 4.12: "Index reference" panel in the Analytics tab, showing stable numeric mappings (e.g., users, storylines, content types, reaction types, and comment identifiers) that can be exported separately to support statistical workflows.

application loads the selected storyline and referenced media from WebDAV and records interactions as structured events in logs.json; at session end, the recorded log is synchronised back to WebDAV. For data extraction, the admin interface reads the stored event stream together with storyline metadata and generates a downloadable CSV export, while the raw log remains available for traceability and potential re-processing.

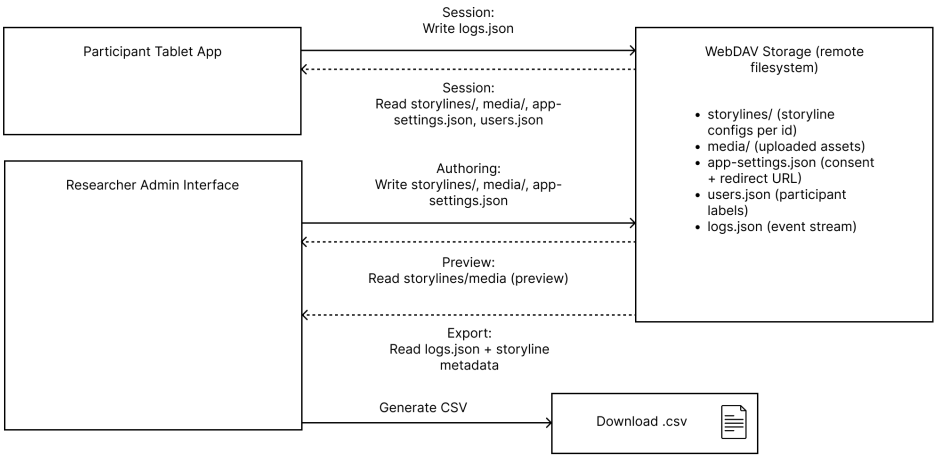


Figure 4.13: Data flow and storage model of the tablet-social-simulator. The admin interface authors storylines, media, and settings; the tablet app reads these artefacts during sessions and appends behavioural events to logs.json; the admin interface generates a CSV export from the stored logs.

4.5.1 File structure

All data is stored as simple files (JSON plus media binaries) under a WebDAV root (Figure 4.14). Conceptually, the backend consists of the following directory tree:

users.json: available participant labels (used for selection / display in sessions)

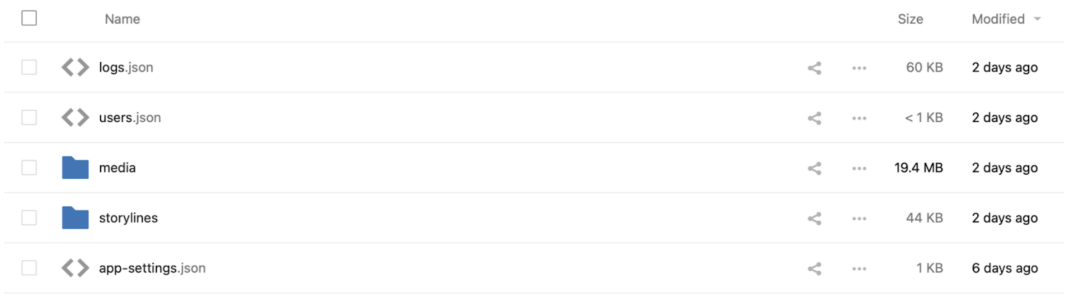


Figure 4.14: WebDAV storage root for the simulator, showing the file-based backend layout with JSON configuration/log files and separate storylines/ and media/ directories.

storylines/<storylineId>.json: storyline configuration per storyline (ordered posts, reactions, comment threads, interaction flags)

media/<filename>: uploaded image/video assets referenced by posts (and avatars, if provided)

logs.json: append-only behavioural event log (one entry per interaction)

app-settings.json: consent and study-level settings editable from the admin UI

This file-oriented layout keeps deployment simple (no database) and makes backups and inspection straightforward.

4.5.2 WebDAV-based cloud storage

The WebDAV backend is accessed exclusively through the data access layer, which is split into two small modules with clear responsibilities:

cloudConfig.js (configuration boundary): centralises deployment-specific settings such as base URLs/paths and consistent URL construction for assets. The key implementation benefit is that backend endpoints are not duplicated across the app and can be changed without touching UI logic.

webdavClient.js (protocol boundary): provides a single, shared client that performs basic WebDAV operations (list/read/write). It hides authentication and request details from the rest of the system, so higher layers treat WebDAV as a filesystem API rather than a networking concern.

On top of the raw WebDAV client, storyService.js implements robustness patterns that reduce common first-run and deployment failure modes:

On-demand directory creation: before writing storylines or media, required folders are ensured to exist (e.g., storylines/, media/).

Safe write semantics (retrying writes after creating missing parent directories), reducing common first-run failure modes.

This preserves separation of concerns: WebDAV protocol details remain in the data-access layer, while persistence policy (what must exist and what gets written) is handled in the domain service.

4.5.3 CSV export

The simulator stores behaviour as a chronological event log (logs.json). The CSV export in AdminPage.js converts this event stream into an analysis-ready table by aggregating events per participant and per post.

1. **Filtering and column schema.** The export is tied to the currently selected storyline (selectedStoryId). Before generating a file, the exporter derives a post metadata list (storylinePostMeta) that determines the number of post columns and assigns a content-type index per post position.

2. **Aggregate logs into a per-user x per-post matrix.** The exporter filters log entries for the selected storyline and builds a record per user with one slot per post:
   - time_on_post events are summed into a total time per post
   - react events are reduced to a single reaction type and encoded via reaction-type indices
   - liked comments are collected and encoded per post

   If a participant has no logged interaction for a post, an empty entry is emitted so the resulting table remains consistent for statistical tools.
3. **Liked comments as numeric indices.** Seeded comments (including replies) are assigned numeric indices based on the storyline's comment structure. When a participant likes a comment, the export stores the corresponding indices as a compact list per post.
4. **Write CSV and download.** Finally, the aggregated rows are serialised into CSV text (header plus rows) and downloaded in the browser.

In addition to the main analytics export, the admin includes exportIndexReferenceCSV(), which outputs a separate reference CSV (codebook-style) containing the index mappings used in the export (e.g., user indices, reaction indices, and comment indices).

4.6 Deployment

4.6.1 Installing the tablet application (Android)

For study sessions, the React frontend is packaged into a native Android container using Capacitor. This results in a permanently installed app with an icon, full-screen behaviour, and standard presence in the Android task switcher. A reproducible installation workflow is:

1. **Build the web application.** [language=bash] npm run build
2. **Synchronise the Capacitor Android project.** [language=bash] npx cap sync android
3. **Install on a tablet via Android Studio.**
   (a) Open the generated Android project in Android Studio
   (b) Connect a tablet (USB debugging enabled)
   (c) Run/install the application as a standard Android app

4.6.2 Deploying the researcher admin interface (web)

The admin interface is deployed locally on the client laptop and runs in a standard web browser.

Deployment steps:

1. Transfer the project folder as a ZIP and extract it
2. Install Node.js on the laptop (if not already installed)
3. Open a terminal in the project root and run: [language=bash] npm install npm start

This starts the React development server and exposes the admin UI in the browser.

4.6.3 WebDAV configurations

Both deployments (tablet app and admin UI) communicate with the same WebDAV storage backend. To avoid hard-coded server details scattered across the codebase, all WebDAV parameters are isolated in the data-access layer. Changing to a new backend server means only this configuration layer needs to be updated.

Chapter 5
Evaluation and Validation

Following the technical implementation, a series of validation steps were conducted to assess the system's effectiveness. The evaluation consisted of: (1) a pilot study with sample participants to assess ecological validity, (2) an expert review with the client to evaluate the administrative workflow, (3) a technical validation of the generated datasets, and (4) a heuristic inspection of both interfaces using Nielsen's 10 usability heuristics.

5.1 Participant Interface Pilot

5.1.1 Methodology

A small sample of test users ($n = 7$), consisting of two psychology professors and five students, completed a short session on an Android tablet using a prepared storyline. The feed shown to participants was the same study setup authored by the client in the admin dashboard for the client evaluation in Section 5.2, and it was intended for an upcoming study. Participants were handed the tablet with the instructions to scroll through the feed for as long as they wanted, to try out the reaction and commenting functions, to click "finish" when they were done, and to pay attention if anything confused them. No additional guidance was provided in order to observe whether the interface was self-explanatory and whether participants would engage with it in a natural manner.

5.1.2 Results and Feedback

After the interaction, participants answered four short questions: (1) how close the experience felt to a real social media app, (2) whether anything distracted them, (3) whether they wanted to do something but could not figure out how, and (4) whether anything felt artificial or "off". The initial reception was uniformly positive, with all participants describing the experience as realistic and expected.

In terms of immersion, one student participant (P5) remarked that the experience felt like "scrolling Facebook in the morning," while a psychology professor (P1) noted that she "expected it to be a real platform" and focused her feedback entirely on the content rather than on software features. Regarding intuitive design, participant P4 summarized the usability by stating, "If you know social media, you know how to use it." All participants navigated the flow from the feed to the end of the session without requiring additional instructions, suggesting that the main interaction patterns were immediately understandable.

More detailed probing also revealed specific gaps between the simulator and expectations formed by commercial platforms. Several participants (P6, P7) attempted to long-press the reaction button to select different emojis reflecting a common interaction pattern on Facebook that was not implemented. Similarly, P3 and P5 commented on the lack of

reaction variety within the comment section, expecting options beyond a basic like. Interestingly, participants also pointed out that the simulation's fidelity was reduced by its "cleanliness." Participant P2 described the experience as "Facebook without ads," and P7 noted that the absence of sponsored content made the feed feel slightly artificial. Finally, one technical issue affected perceived smoothness: participant P2 observed that image loading times immediately after the consent screen were noticeable, briefly breaking immersion before the feed fully rendered.

5.2 Client Evaluation: Admin Dashboard

5.2.1 Methodology

The client evaluated the admin dashboard in the context of preparing an upcoming study. After a short introduction to the interface, the client independently authored a complete storyline consisting of 18 posts, using the available editing and preview functionality. Observations were noted during the session and followed by a short semi-structured interview focusing on perceived intuitiveness, workflow fit, and any points of confusion or friction.

5.2.2 Interview Findings and Feedback

In the interview, the client described the dashboard as "very intuitive." This was also reflected in observed behaviour during the authoring session. The client used the Preview tab on their own to check the feed after editing, without being instructed to do so, indicating that the interface made verification workflows easy to discover and consistent with their expectations. The client further emphasized that the separation between content authoring and interaction logic supported fast iteration, allowing them to refine the storyline structure and details without technical support.

At the same time, the session revealed several areas for refinement. When adding a nested comment in Live Preview mode, the client encountered a non-blocking error message; while the feature still worked, the presence of an unexplained error reduced confidence and should be addressed to avoid uncertainty during study preparation. In addition, the client occasionally forgot to manually save changes, suggesting that saving is not sufficiently salient in the natural workflow and motivating an auto-save mechanism or a clear "unsaved changes" warning. Finally, although post creation was perceived as straightforward, the client required additional guidance to understand the logic for adding replies to comments, indicating that this part of the interaction model would benefit from clearer affordances or brief inline explanations.

5.3 Data Output

A key measure of the simulator's utility is the quality and usability of the data it produces for statistical analysis.

Raw telemetry (JSON logs)

The system records behavioural events as JSON entries. Each event contains a timestamp, participant identifier, storyline reference, and a post index, which makes individual interactions traceable and allows behaviour to be mapped to specific stimuli. The event schema distinguishes passive engagement (e.g., dwell time per post) from active engagement (e.g., reactions), enabling separate analyses of exposure and interaction. An excerpt is shown in Figure 5.1.

Figure 5.1: Raw JSON telemetry excerpt showing an event-level "react" action and a subsequent "time_on_post" entry with millisecond dwell time.

For reference, the excerpt contains events of the following form:


```
{
  "ts": "2025-12-12T14:23:03.934Z",
  "storylineId": "upward-comparisons",
  "storylineName": "Test",
  "user": "Max",
  "action": "react",
  "postIndex": 17,
  "value": { "type": "haha" }
}

{
  "ts": "2025-12-12T14:23:08.912Z",
  "storylineId": "upward-comparisons",
  "storylineName": "Test",
  "user": "Max",
  "action": "time_on_post",
  "postIndex": 17,
  "value": { "durationMs": 18443 }
}
```


Exported dataset (CSV)

For analysis, the JSON event logs are transformed into a wide-format CSV file that can be imported into common environments (Excel, SPSS, R, Python). Each row represents one participant session, while columns represent per-post metrics (e.g., reaction codes, time spent, and like state), allowing straightforward statistical modelling across participants and conditions. To support quantitative workflows, the aggregated output contains numerical values only. Figure 5.2 illustrates an excerpt of the generated wide-format CSV structure. This export format validates that the simulator's telemetry can be converted into a directly usable dataset for standard statistical workflows, reducing post-processing effort and supporting reproducible analysis across studies.

userIndex	storylineInde	post1_conter	post1_reacti	post1_timeS	post1_likedC	post2_conter
7	4	1	4	215		2
2	4	1	0	39		2
6	4	1	0	23		2
9	4	1	0	104		2
3	4	1	3	29		2
8	4	1	0	189		2
4	4	1	0	114		2

Figure 5.2: Excerpt of the wide-format CSV export. Each row represents one participant session (userIndex), and columns contain per-post metrics (e.g., reaction code and time spent) for direct import into statistical analysis tools.

5.4 Heuristic evaluation (Nielsen's 10 usability heuristics)

To complement the user- and client-based evaluation in Sections 5.1–5.3, both interfaces were inspected using Nielsen's 10 usability heuristics. The inspection focused on the most relevant heuristics for the simulator's two primary user groups (participants and researchers).

5.4.1 Participant-facing tablet interface

Heuristic 1 (Visibility of system status) The participant interface provides clear status feedback through explicit loading indicators for pending storylines/media and immediate UI updates after interactions such as likes/reactions and opening or closing comments. Progress is also transparent as primary actions are disabled until required inputs are complete (participant name/storyline selection and explicit consent). A lightweight step indicator and unobtrusive "saving/saved" plus offline/network cues would improve status clarity without drawing attention away from the content.

Heuristic 2 (Match between system and the real world) The `StoryViewer` adopts a familiar Facebook-style feed model (posts, reaction bar, comments, replies) and uses plain labels ("Continue," "Finish," "Start survey"), supporting naturalistic use without training. The key realism risk is where expected platform conventions are simplified, such as richer reaction mechanics or typical ambient feed elements. These gaps do not block use, but they can affect perceived authenticity and thus behaviour.

Heuristic 3 (User control and freedom) User control is supported through reversible interactions (like/reaction toggles) and cancellable composition for comments and replies. Navigation decisions are explicit at key points, including consent and the end-of-story "Finish" action. A limitation is that submitted comments cannot be undone or deleted, reducing recovery from accidental submissions.

Heuristic 5 (Error prevention) The interface prevents common mistakes by enforcing required inputs before users can proceed, requiring explicit consent, and blocking empty comment or reply submissions. The remaining risks are typical tablet "slip" errors: double-tapping during lag can submit actions twice, the survey launch can be triggered accidentally, and typed drafts can be lost if users navigate away mid-entry. These risks can be reduced by temporarily disabling buttons while a save is in progress, adding a simple confirmation step before starting the survey, and prompting users before leaving when a draft is open.

Heuristic 9 (Help users recognize, diagnose, and recover from errors) Robustness measures such as caching/prefetching and fallback behaviour reduce the likelihood that media failures break layouts or block sessions. When saving fails (e.g., comment persistence), explicit error messages make failures visible, but interruptive alerts can break immersion and potentially affect behaviour. A more study-friendly recovery approach is to surface offline/network state proactively and handle failed saves non-blockingly via retry/queue patterns, preserving flow while maintaining data integrity.

Overall, the participant interface aligns most strongly with Heuristic 1 (Visibility of system status) and Heuristic 3 (User control and freedom) through explicit loading cues, immediate interaction feedback, and reversible/cancellable actions, supporting a smooth and realistic participant experience. The main improvement opportunities concern Heuristic 5 (Error prevention) and Heuristic 9 (Help users recognize, diagnose, and recover from errors): reducing tablet slip errors and improving the handling of connectivity failures would improve robustness and protect data quality without weakening experimental control.

5.4.2 Researcher admin dashboard

Heuristic 1 (Visibility of system status) In the settings view, loading and saving are communicated through visible status text (e.g., "Loading settings...") and a persistent status message that confirms success or shows an error. In the storyline editor, the save button switches into a saving state and success or failure is communicated to the user after the operation completes. Destructive actions such as deleting a storyline also provide completion feedback. Non-blocking progress indicators next to the relevant buttons (save/delete/export) and a brief success message that fades would make progress and outcomes easier to interpret.

Heuristic 3 (User control and freedom) Researchers can cancel editing to avoid committing unintended changes, and posts, comments, and replies can be removed during drafting, enabling quick correction. Storylines can be duplicated or deleted, supporting iterative development, and analytics functions allow logs to be cleared to recover from test runs.

Heuristic 4 (Consistency and standards) The preview uses the same feed/card patterns as the participant view, helping researchers reliably anticipate what participants will see, while core controls (buttons, inputs, toggles) follow consistent styling across tabs. Unifying feedback (e.g., always using the same inline/toast notification pattern) would make the interface more predictable and reduce mistakes.

Heuristic 5 (Error prevention) Storylines cannot be saved without required fields (e.g., a name and at least one post), destructive actions require confirmation, and actions are disabled when the relevant context is not ready (for example, while settings are loading/saving). The interface also applies safe defaults that prevent invalid configurations, such as default reaction settings so they cannot become empty. The main improvement is to show validation directly where errors occur (inline, next to the affected field) rather than relying on generic alerts, and to add "unsaved changes" protection when switching storylines or tabs.

Heuristic 6 (Recognition rather than recall) The builder and preview reduce memory load by keeping the current storyline visible as a feed, so researchers can verify what has been authored without recalling details. The stepwise authoring flow further supports recognition by keeping relevant controls in view when needed. This can be strengthened with small context cues, such as showing media thumbnails in the composer and labeling required next actions inline (e.g., "Step 2: Add your first post").

Heuristic 9 (Help users recognize, diagnose, and recover from errors) Previewing storylines supports recovery by letting researchers catch configuration mistakes before running participant sessions. When actions fail (e.g., saving settings or storylines), the system provides error feedback so failures are not silent and users can retry. A remaining weakness is that some messages are generic or interruptive and not always shown next to the triggering control, which slows recovery. More actionable, in-context messages would improve diagnosis without disrupting the workflow.

Overall, the admin dashboard is strongest on Heuristic 3 (User control and freedom), Heuristic 5 (Error prevention), and Heuristic 6 (Recognition rather than recall). They support the project goals of strong experimental control and an authoring workflow that non-technical researchers can use independently. The main improvements relate to Heuristic 1 (Visibility of system status) and Heuristic 9 (Help users recognize, diagnose, and recover from errors): clearer non-blocking progress feedback and more actionable, in-context error messages would further reduce configuration mistakes and protect data integrity.

Chapter 6
Conclusions

This project addressed a practical gap in psychological research tooling: enabling realistic, social-media-like interaction while preserving deterministic stimulus control, fine-grained behavioural logging, and privacy-preserving storage under institutional control. The resulting tablet-social-simulator combines an Android tablet participant application with a researcher-facing admin dashboard for authoring storylines, previewing the participant experience, configuring consent and survey handoff, and exporting analysis-ready datasets.

6.1 Review of project goals

Within the defined scope, the project achieved the seven core goals (G1–G7) established in the problem formulation.

G1 & G2: Participant experience and ecological validity

The participant interface supports a complete session flow on Android tablets and reproduces core feed interaction patterns (scrolling, reacting, and engaging with comment threads). In the participant pilot, users were able to complete the session without additional guidance and described the interaction logic as familiar and intuitive, indicating a strong baseline of ecological validity for the intended use. In terms of the Technology Readiness Level framework [**?**], the system has reached TRL 6: it has been demonstrated in a relevant environment with representative content and real participant interaction on the target hardware. Advancing to TRL 7 and beyond would require repeated, independent deployments by multiple research groups across different study designs.

G3 & G4: Researcher workflow and experimental control

The admin dashboard supports authoring and configuration of storylines and study settings, complemented by a preview mechanism that allows researchers to verify the participant experience before deployment. The client evaluation suggests that the workflow is learnable and supports iterative preparation without code changes. Experimental control is ensured through researcher-defined stimulus ordering and configuration choices that keep exposure reproducible across sessions. The evaluation also highlighted workflow refinements (notably save-state clarity and error feedback) that would further reduce preparation mistakes under realistic conditions.

G5–G6: Behavioural logging and analysis-ready export

The system captures structured behavioural events during sessions, covering both active interactions (e.g., reactions) and passive engagement measures (e.g., viewing/dwell signals), and links them to feed items via stable identifiers and indices. The export functionality transforms the event stream into a structured dataset suitable for statistical workflows, reducing manual post-processing. Technical validation indicates that the logging and export pipeline produces coherent, analysis-ready outputs aligned with the project's emphasis on data quality and interpretability.

G7: Privacy and institutional control

Study assets and behavioural logs are stored as simple files on researcher-controlled infrastructure via a WebDAV-backed filesystem model. This keeps deployment transparent (no database) and supports backups and inspection while aligning with privacy and governance constraints.

6.2 Summary of contributions

A lab-ready, tablet-focused experimental platform

The simulator provides a dedicated Android tablet participant interface that reproduces core feed interaction patterns in a controlled environment suited for in-person lab studies. Pilot results suggest that participants can use the interface naturally with minimal instruction, placing the system at TRL 6 [**?**].

An accessible authoring and verification workflow for non-technical researchers

The admin dashboard enables study preparation without source code modification: storylines can be authored, previewed, and iterated, and global study texts (consent and survey handoff) can be configured directly in the interface. Client feedback indicates that key workflows are discoverable and match researcher expectations.

A storage and export pipeline aligned with institutional control and statistical analysis

The file-oriented WebDAV backend provides transparent persistence that is easy to administer under university hosting constraints. The analytics/export pipeline converts the event logs into a structured dataset with stable numeric encodings and a separate indices reference, supporting reproducible analysis across studies.

6.3 Critical reflection and limitations

The simulator achieves convincing feed interaction within a controlled design, but several factors define the boundaries of its applicability.

Ecological validity versus experimental control

The simulator deliberately prioritises deterministic exposure over full platform realism. Key delimitations, such as no recommendation algorithms, no dynamic ranking or personalisation, and no full social-network features, support reproducible stimulus presentation but also mean that behaviour may differ from real platforms. The tool should therefore be understood as a controlled approximation of feed interaction patterns rather than a full replication of commercial social media.

Interpretation limits of behavioural telemetry

While the logging and export pipeline produces analysis-ready measures, the variables remain behavioural proxies. For example, dwell/time-on-post estimates exposure duration, not attention or comprehension, and reactions/comments reflect observable engagement rather than attitudes. Most study designs will therefore require combining simulator logs with external instruments (e.g., surveys or manipulation checks) to support psychological interpretation.

Workflow robustness and configuration risk

The study setup process still depends on correct researcher actions (e.g., saving after edits, validating intended interaction constraints). Under time pressure, these human factor risks can become relevant. Future iterations should reduce avoidable misconfiguration (e.g., clearer save-state signalling and more defensive validation).

Evaluation scope and generalisability

The evaluation was designed as a practical validation, not a large-scale usability study. Broader claims about usability, realism, or behavioural generalisation would require larger and more diverse participant samples and repeated deployments across varied conditions.

Implication for use

The simulator is well-suited for psychological studies that require controlled feed-based exposure, systematic manipulation of social signals, and structured behavioural logging under institutional control. It is less suited for research questions that depend on algorithmic feed dynamics, long-term social network effects, or fully naturalistic platform complexity.

6.4 Future work

Future iterations should build on deterministic stimulus delivery, structured logging, and an accessible researcher workflow. The main objective is to increase ecological validity and broaden applicability to more study designs, while preserving experimental control and interpretability of the exported data.

Short-term refinements

First, the participant-side interaction model can be made more platform-authentic by introducing richer reaction mechanics, including long-press reaction selection and an expanded reaction set that is also available within comment threads. Second, realism can be increased without sacrificing determinism by introducing optional ambient feed elements such as sponsored or neutral filler posts that can be enabled per storyline or experimental condition. Third, the researcher workflow should be strengthened through UI refinements that reduce avoidable configuration errors, including autosave or a clear unsaved-changes indicator, improved reply-to-comment authoring affordances, and more actionable, in-context error messages. In addition, replacing interruptive alert dialogs with study-friendly, non-blocking error handling (including retry/queue behaviour for transient connectivity issues) would improve robustness during both study preparation and participant sessions. Finally, the session flow could benefit from a dedicated study explanation page that standardises participant instructions and reduces the need for external briefing, improving consistency across study runs.

Longer-term extensions

The admin interface could expand the simulator beyond the current platform constraints by developing versions optimised for smartphones and/or a cross-platform web deployment, enabling more flexible study setups and potentially more naturalistic participant contexts. In parallel, adding optional simulations of platform dynamics, such as notifications or controlled dynamic social feedback where feed elements respond to participant actions, could increase realism for specific research questions, provided these mechanisms remain explicitly configurable and reproducible. Finally, broader support for platform skins (e.g., alternative visual styles and interaction conventions inspired by other social media platforms) would allow researchers to match the simulator to the context most relevant for their target population while keeping the underlying data model consistent.

References

[1] J. R. Anthis et al. LLM Social Simulations Are a Promising Research Method. arXiv preprint arXiv:2504.02234, 2025. (Not cited.)

[2] Association of Internet Researchers (AoIR). Internet Research: Ethical Guidelines 3.0. AoIR, 2020. (Cited on pages 9 and 10.)

[3] J. Breuer, S. Stier, J. Lukito, F. Mangold, M. Wieland, and D. Radovanović. Overview of Ethical Considerations when Working with Digital Behavioral Data. GESIS Guides to Digital Behavioral Data, No. 14, 2025. DOI: 10.60762/ggdbd25014.1.0. (Cited on pages 9 and 10.)

[4] D. Bruns, J. F. Kopka, and L. Borgmann. Measuring Gaining and Holding Attention to Social Media Ads with Viewport Logging: A Validation Study Using Mobile Eye-Tracking. Journal of Advertising, 2025. DOI: 10.1080/00913367.2025.2524186. (Not cited.)

[5] L. H. Butler et al. The (Mis)Information Game: A social media simulator. Behavior Research Methods, 2023. DOI: 10.3758/s13428-023-02153-x. (Cited on page 9.)

[6] European Data Protection Board (EDPB). Legal study on the appropriate safeguards under Article 89(1) GDPR for the processing of personal data for scientific research. EDPB, 2022. (Cited on page 10.)

[7] European Data Protection Board (EDPB). Summary of Guidelines 01/2025 on Pseudonymisation. EDPB, 2025. (Cited on page 10.)

[8] Frontiers. Is your social media research ethical? Here's what you need to know. Frontiers Field Notes, 2025. (Cited on page 8.)

[9] A. Jagayat, G. Boparai, C. Pun, and B. L. Choma. Mock Social Media Website Tool (MSMWT) (1.0) [Computer software], 2021. (Cited on page 9.)

[10] A. Jagayat and B. L. Choma. A primer on open-source, experimental social media simulation software: Opportunities for misinformation research and beyond. Current Opinion in Psychology, 55:101726, 2024. DOI: 10.1016/j.copsyc.2023.101726. (Cited on pages 8 and 9.)

[11] A. D. I. Kramer, J. E. Guillory, and J. T. Hancock. Experimental evidence of massive-scale emotional contagion through social networks. Proceedings of the National Academy of Sciences (PNAS), 111(24):8788–8790, 2014. DOI: 10.1073/pnas.1320040111. (Cited on page 10.)

[12] D. Lagun and M. Lalmas. Understanding and Measuring User Engagement and Attention in Online News Reading. In Proceedings of the Ninth ACM International Conference on Web Search and Data Mining (WSDM '16), 2016. DOI: 10.1145/2835776.2835833. (Cited on page 10.)

[13] M. Larooij and P. Törnberg. Can We Fix Social Media? Testing Prosocial Interventions using Generative Social Simulation. arXiv preprint arXiv:2508.03385, 2025. (Not cited.)

[14] M. Larooij and P. Törnberg. Validation is the central challenge for generative social simulation: a critical review of LLMs in agent-based modeling. Artificial Intelligence Review, 59(1):15, 2026. DOI: 10.1007/s10462-025-11412-6. (Cited on page 10.)

[15] A. T. Mayer, J. Ohme, et al. Headlines, Pictures, Likes: Attention to Social Media Newsfeed Post Elements on Smartphones and in Public. Social Media + Society, 10(2), 2024. DOI: 10.1177/20563051241245666. (Not cited.)

[16] The Menlo Report Authors. The Menlo Report: Ethical Principles Guiding Information and Communication Technology Research. 2012. (Cited on page 10.)

[17] National Commission for the Protection of Human Subjects of Biomedical and Behavioral Research. The Belmont Report: Ethical Principles and Guidelines for the Protection of Human Subjects of Research. 1979. (Cited on page 10.)

[18] L. H. X. Ng and K. M. Carley. Are LLM-Powered Social Media Bots Realistic? arXiv preprint arXiv:2508.00998, 2025. (Not cited.)

[19] D. A. Parry, B. I. Davidson, C. J. R. Sewall, J. T. Fisher, H. Mieczkowski, and D. S. Quintana. A systematic review and meta-analysis of discrepancies between logged and self-reported digital media use. Nature Human Behaviour, 5(11):1535–1547, 2021. DOI: 10.1038/s41562-021-01117-5. (Cited on page 8.)

[20] R. G. Sargent. Verification and validation of simulation models. Journal of Simulation, 7(1):12–24, 2013. DOI: 10.1057/jos.2012.20. (Cited on page 10.)

[21] V. J. Straub et al. Public attitudes towards social media field experiments. Scientific Reports, 14:26110, 2024. DOI: 10.1038/s41598-024-76948-z. (Cited on page 10.)

[22] Cornell Social Media Lab. The Truman Platform (project page), n.d. (Cited on page 9.)

[23] P. Törnberg and D. Valeeva. Simulating Social Media Using Large Language Models to Evaluate Alternative News Feed Algorithms. arXiv preprint arXiv:2310.05984, 2023. (Not cited.)

[24] T. Verbeij, J. L. Pouwels, I. Beyens, and P. M. Valkenburg. The accuracy and validity of self-reported social media use measures among adolescents. Computers in Human Behavior Reports, 3:100090, 2021. DOI: 10.1016/j.chbr.2021.100090. (Cited on page 8.)

[25] I. M. Verma. Editorial Expression of Concern: Experimental evidence of massive-scale emotional contagion through social networks. Proceedings of the National Academy of Sciences (PNAS), 111(29):10779, 2014. DOI: 10.1073/pnas.1412469111. (Cited on page 10.)

[26] J. C. Mankins. Technology Readiness Levels: A White Paper. NASA, Office of Space Access and Technology, 1995. (Not cited.)